



Université Mohammed V de
Rabat

École Nationale Supérieure
d'Arts et Métiers — ENSAM
Rabat

Filière Ingénierie Numérique en Data
Science
et Intelligence Artificielle — 2^{ème} année

Année Universitaire 2025/2026



Projet de Fin d'Année

Tamwil AI

Chatbot RAG de recommandation
financière pour startups

*Un assistant conversationnel combinant
Retrieval-Augmented Generation et règles métier
au service de l'écosystème startup marocain et francophone*

Réalisé par :

Aymané ISSAMI

Houssam KICHCHOU

Filière INDIA — 2^{ème} année

Encadré par :

M^{me} Hafsa KHIYAT

Professeurs responsables :

M^{me} Amal TMIRI

M^{me} Soukaina BEKKARI

Soutenu publiquement le _____

À nos familles,

*pour leur soutien indéfectible tout au long
de notre parcours scolaire et universitaire.*

À nos enseignants,

*dont la rigueur et la passion nous ont donné
le goût du travail bien fait.*

À l'écosystème entrepreneurial marocain,

*qui nous a inspirés ce projet,
dans l'espoir qu'il lui soit utile.*

Remerciements

Au terme de ce projet, nous tenons à exprimer notre profonde reconnaissance à toutes les personnes qui ont contribué, de près ou de loin, à son aboutissement.

Nous adressons nos remerciements les plus sincères à notre encadrante, Madame **Hafsa Khiyat**, pour son suivi rigoureux, la qualité de ses conseils, la pertinence de ses remarques et la disponibilité dont elle a fait preuve tout au long de ce travail. Ses retours détaillés sur notre première version ont été déterminants pour la qualité de ce rapport.

Nos remerciements vont également à Madame **Amal Tmiri** et à Madame **Soukaina Bekkari**, professeures responsables, pour la coordination du projet et pour nous avoir permis de travailler sur un sujet en prise directe avec les enjeux économiques de notre pays.

Nous remercions chaleureusement l'ensemble du corps professoral de l'**École Nationale Supérieure d'Arts et Métiers de Rabat** (ENSAM), de la filière *Ingénierie Numérique en Data Science et Intelligence Artificielle*, pour la formation de qualité qui nous a permis d'acquérir les compétences techniques et méthodologiques mobilisées dans ce travail.

Nous exprimons enfin notre gratitude à nos familles, pour leur soutien moral constant, et à nos camarades de promotion, pour les échanges enrichissants tout au long de l'année.

Que toutes et tous trouvent ici l'expression de notre sincère reconnaissance.

Résumé

Les fondateurs de startups au Maroc et en Afrique francophone font face à un manque d'information financière structurée et adaptée à leur contexte. Les réponses aux questions essentielles — quel financement choisir, quels investisseurs cibler, suis-je prêt à lever des fonds — sont dispersées, souvent en anglais et peu personnalisées.

Ce projet, **Tamwil AI**, propose un assistant conversationnel fondé sur la technique *Retrieval-Augmented Generation* (RAG). Le système combine une base de connaissances constituée manuellement (41 profils d'investisseurs, 25 programmes de subventions, 18 indicateurs clés de performance avec *benchmarks* marocains, 54 questions-réponses et dix documents longs sur le financement et la réglementation) et un grand modèle de langue (GPT-3.5-turbo) pour produire des réponses contextuelles en français.

L'architecture est volontairement hybride : un pipeline RAG vectoriel (embeddings multilingues locaux, ChromaDB persistant) traite les questions ouvertes, tandis que des modules déterministes — *scoring* de *fundability*, diagnostic financier, matching d'investisseurs et de subventions — exploitent les données structurées selon des règles explicites. Un routeur à base de mots-clés et d'expressions régulières oriente chaque requête vers le module approprié.

Le backend est développé en Python avec FastAPI et expose une API REST comprenant un flux *Server-Sent Events* pour le *streaming* de la génération. L'interface utilisateur repose sur Next.js 16, React 19 et Tailwind CSS 4 ; elle offre une expérience proche de celle des assistants conversationnels grand public (barre latérale avec historique, mode sombre, formulaire de profil, visite guidée pour les nouveaux utilisateurs).

Le système a été validé par six scénarios fonctionnels et une batterie de tests unitaires couvrant tous les modules métier et le pipeline RAG. Les limites identifiées — dataset à enrichir, détection d'intention par mots-clés à remplacer par une classification par LLM, persistance serveur de l'historique — ouvrent la voie à des évolutions futures.

Mots-clés : RAG, LLM, chatbot, financement de startup, ChromaDB, LangChain, *embeddings* multilingues, FastAPI, Next.js, Maroc.

Abstract

Startup founders in Morocco and French-speaking Africa face a structural lack of financial information tailored to their context. Answers to core questions — which funding to choose, which investors to target, am I ready to raise — are scattered, mostly in English, and rarely personalised.

This project, **Tamwil AI**, proposes a conversational assistant based on the *Retrieval-Augmented Generation* (RAG) technique. It combines a hand-curated knowledge base (41 investor profiles, 25 grant programmes, 18 startup KPIs with Moroccan benchmarks, 54 Q&A, and ten long documents on fundraising and regulation) with a large language model (GPT-3.5-turbo) to produce contextual French answers.

The architecture is deliberately hybrid : a vector RAG pipeline (multilingual local embeddings, persistent ChromaDB) handles open-ended questions, while deterministic modules — fundability scoring, financial diagnosis, investor and grant matching — operate on structured data through explicit rules. A keyword-and-regex router dispatches each query to the right module.

The backend is a Python/FastAPI service exposing a REST API with a Server-Sent Events stream for token-level generation. The user interface is a Next.js 16 / React 19 / Tailwind CSS 4 application, offering a modern chat experience (collapsible sidebar with conversation history, dark mode, startup profile form, guided tour).

The system has been validated through six functional scenarios and a unit-test suite covering every business module and every stage of the RAG pipeline. The identified limits — dataset size, keyword-based intent detection, client-side-only history — are discussed as future work.

Keywords : RAG, LLM, chatbot, startup funding, ChromaDB, LangChain, multilingual embeddings, FastAPI, Next.js, Morocco.

Table des matières

Remerciements	ii
Résumé	iii
Abstract	iv
Liste des acronymes	x
Introduction générale	1
1 Étude de l'existant et cadre théorique	5
1.1 Écosystème du financement des startups	5
1.1.1 Panorama et enjeux	5
1.1.2 Acteurs du financement	5
1.2 Assistants conversationnels : bref panorama	6
1.3 La technique Retrieval-Augmented Generation	6
1.3.1 Principe	6
1.3.2 Pipeline détaillé	6
1.3.3 Apports du RAG par rapport au <i>fine-tuning</i>	7
1.4 Positionnement par rapport aux solutions existantes	8
1.5 Justification de l'approche hybride	8
2 Conception et architecture	9
2.1 Architecture globale	9
2.2 Modules fonctionnels	10
2.3 Modèle de données	11
2.4 Pipeline RAG	11
2.5 Mapping dataset → fonctionnalités	12
3 Réalisation et implémentation	13
3.1 Environnement	13
3.2 Stack technique	13
3.3 Constitution du dataset	14
3.4 Pipeline RAG	14
3.4.1 Configuration centralisée	14
3.4.2 Chargement des données	15
3.4.3 Ingestion et chunking	15
3.4.4 Modèle d'embedding	15
3.4.5 Base vectorielle et recherche	15
3.5 Modules métier	15
3.5.1 Scoring de <i>fundability</i>	15
3.5.2 Diagnostic financier	16
3.5.3 Matching investisseurs et subventions	16
3.5.4 Q&A conversationnel	16
3.6 Backend FastAPI	16

3.6.1	Endpoints	16
3.6.2	Modèles Pydantic et normalisation du profil	17
3.6.3	Détection de <i>greetings</i>	17
3.6.4	Routeur d'intentions	17
3.6.5	Streaming SSE	18
3.6.6	Attribution des sources et formatage	18
3.7	Interface Next.js	19
3.7.1	Architecture des composants	19
3.7.2	Barre latérale et gestion de conversations	19
3.7.3	Chat et rendu Markdown	19
3.7.4	Formulaire de profil	19
3.7.5	Onboarding et thèmes	19
3.7.6	Gestion d'état et consommation du flux SSE	20
3.8	Structure du projet	20
3.9	Règles de développement	21
4	Tests et résultats	22
4.1	Scénarios fonctionnels	22
4.2	Validation du routeur d'intentions	24
4.3	Tests du pipeline RAG	28
4.3.1	Optimisation de Recall@K	28
4.4	Performance de l'API de streaming	30
4.4.1	Choix de GPT-3.5	32
4.5	Tests unitaires	33
4.6	Discussion et limites	33
4.6.1	Points forts	33
4.6.2	Limites identifiées	34
5	Conclusion générale et perspectives	35
5.1	Bilan du projet	35
5.2	Apports personnels	35
5.3	Perspectives d'amélioration	36
	Bibliographie et webographie	38
A	Schéma JSON du dataset	40
B	Variables d'environnement	43
C	Référence API FastAPI	44
D	Prompt système du module Q&A	46

Table des figures

1	Planning prévisionnel et réel du projet Tamwil AI, de février à avril 2026. Les phases 2 à 5 ont été réalisées sur cinq semaines (commits Git entre le 10 février et le 25 mars 2026), la phase 6 couvre la rédaction et l'itération avec l'encadrante jusqu'à la soutenance.	3
1.1	Pipeline RAG en six étapes. Les trois premières (ingestion, <i>chunking</i> , <i>embedding</i>) sont exécutées une fois lors de la construction de l'index ; les trois suivantes (indexation, <i>retrieval</i> , génération) s'exécutent à chaque requête utilisateur.	7
2.1	Architecture globale de Tamwil AI — trois couches (interface Next.js, service FastAPI avec routeur et modules, données ChromaDB + fichiers). Les flèches en pointillé représentent les flux optionnels (API externe, module porté par le routeur).	9
3.1	Diagramme de séquence d'un échange Q&A en <i>streaming</i> . Après la phase de <i>retrieval</i> dans ChromaDB, le routeur relaie la requête au Large Language Model (LLM) et retransmet chaque <i>token</i> reçu à l'interface via un flux Server-Sent Events (SSE).	18
3.2	(a) Mode sombre	20
3.3	(b) Mode clair	20
3.4	Interface d'accueil de Tamwil AI en mode sombre et en mode clair. La barre latérale affiche l'historique des conversations ; la zone centrale présente un message de bienvenue et quatre cartes de suggestion (score de <i>fundability</i> , recherche d'investisseurs, subventions, diagnostic financier).	20
4.1	Scénario 1 — scoring de <i>fundability</i> . Requête : « <i>Quel est mon score ? MRR : 8000, burn : 15000, churn : 5%, stade seed</i> ». Le module retourne un score pondéré de 42/100, une ventilation par métrique avec pastilles de niveau (<i>ok/bad</i>), une liste de faiblesses et un plan d'action en deux points.	23
4.2	Scénario 3 — matching investisseurs. Profil saisi : <i>fintech, seed, Maroc</i> . Le module identifie 41 investisseurs du dataset et restitue les cinq premiers (212 Founders, Azur Innovation Fund, Flat6Labs, Kalys Ventures, MITC Capital) avec <i>ticket</i> , score de <i>matching</i> (3/4 pour tous ici) et raisons détaillées.	23
4.3	Scénario 6 — Q&A Retrieval-Augmented Generation (RAG). Question : « <i>Comment valoriser ma startup en pré-seed ?</i> ». La réponse, générée en <i>streaming</i> , présente cinq méthodes de valorisation tirées du guide de <i>fundraising</i> du dataset. Les quatre pastilles numérotées en bas (<i>pitch deck, types financement, term sheet, etapes levee</i>) sont les sources cliquables attribuées par le moteur de <i>retrieval</i>	24
4.4	Matrice de confusion du routeur d'intentions, évaluée sur 200 requêtes représentatives.	25
4.5	Synthèse visuelle des métriques du routeur : accuracy, F1-score moyen, erreurs totales et détail par classe.	27
4.6	Recall@K du pipeline RAG pour $K = 1, 3, 5, 10$	30

4.7	Distribution des latences de l'Application Programming Interface (API) en <i>streaming</i> , mesurée sur 100 requêtes.	32
-----	--	----

Liste des tableaux

1.1	Les six étapes du pipeline RAG de Tamwil AI.	7
1.2	Positionnement de Tamwil AI par rapport aux solutions existantes. . . .	8
2.1	Composition du dataset de Tamwil AI.	11
2.2	Mapping entre sources de données et fonctionnalités exposées.	12
3.1	Environnement de développement.	13
3.2	Stack technique et justifications.	14
3.3	Endpoints exposés par le serveur FastAPI.	17
3.4	Intentions reconnues par le routeur et déclencheurs typiques.	17
4.1	Scénarios fonctionnels validés manuellement.	22
4.2	Synthèse de validation du routeur d'intentions.	26
4.3	Métriques de classification par classe pour le routeur.	26
4.4	Résultats de l'ingestion sur le dataset cible.	28
4.5	Recall@K du pipeline RAG sur la base ChromaDB.	29
4.6	Distribution des latences mesurées sur 100 requêtes.	31
4.7	Fichiers de tests unitaires et périmètre couvert.	33
A.1	Composition détaillée du dataset par type d'acteur.	42
A.2	Guides de <i>fundraising</i> disponibles en annexe du dataset.	42
A.3	Documents réglementaires disponibles en annexe du dataset.	42
B.1	Variables d'environnement et leur rôle.	43

Liste des acronymes

API	Application Programming Interface
ARR	Annual Recurring Revenue
BA	Business Angel
BAM	Bank Al-Maghrib
CAC	Customer Acquisition Cost
CDG	Caisse de Dépôt et de Gestion
CNDP	Commission Nationale de contrôle de la protection des Données à caractère Personnel
CORS	Cross-Origin Resource Sharing
CRM	Customer Relationship Management
CVC	Corporate Venture Capital
DAU	Daily Active Users
DCF	Discounted Cash Flow
ESOP	Employee Stock Option Plan
FAQ	Foire Aux Questions
GMV	Gross Merchandise Value
GPU	Graphics Processing Unit
INDIA	Ingénierie Numérique en Data Science et Intelligence Artificielle
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
KYC	Know Your Customer
LLM	Large Language Model
LTV	Lifetime Value
MAD	Moroccan Dirham
MAU	Monthly Active Users
MENA	Middle East and North Africa
MRR	Monthly Recurring Revenue
NLP	Natural Language Processing
NLU	Natural Language Understanding
NPS	Net Promoter Score
NR	Net Revenue Retention
PFA	Projet de Fin d'Année
PME	Petite et Moyenne Entreprise
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer

RGPD	Règlement Général sur la Protection des Données
SaaS	Software as a Service
SARL	Société À Responsabilité Limitée
SENTENCE-BERT	Sentence Bidirectional Encoder Representations from Transformers
SSE	Server-Sent Events
TPV	Total Payment Volume
VC	Venture Capital

Introduction générale

Contexte

L'écosystème entrepreneurial du Maroc et de l'Afrique francophone connaît depuis quelques années une dynamique remarquable. Selon les rapports annuels de *Partech Africa* et de la Banque mondiale, le volume de financement levé par les startups africaines a atteint des niveaux records, porté par la maturation d'un réseau local d'acteurs — Venture Capital (VC), fonds publics, accélérateurs, Business Angel (BA). Le Maroc, en particulier, se distingue par l'émergence de véhicules d'investissement structurants tels que *CDG Invest*, *212 Founders* ou le *Mohammed VI Fund*, et par des dispositifs publics dédiés comme *Innov Invest* ou *FORSA*.

Malgré cette dynamique, les fondateurs de startups restent confrontés à un obstacle pratique majeur : **l'accès à une information financière structurée, actualisée et adaptée au contexte local** est difficile. Les questions concrètes qu'un entrepreneur se pose — « Quel financement est adapté à mon stade ? », « Quels investisseurs ciblent mon secteur au Maroc ? », « Suis-je prêt à lever des fonds ? », « Quelles aides publiques puis-je mobiliser ? » — n'ont pas aujourd'hui de réponse centralisée.

L'information utile est en effet fragmentée entre sites officiels, rapports Portable Document Format (PDF), articles de presse, profils *LinkedIn* et réseaux informels. Elle est fréquemment rédigée en anglais, peu alignée sur le cadre juridique et fiscal marocain, et rarement comparative ou personnalisable.

Problématique

Comment concevoir un système intelligent capable de **centraliser, structurer et personnaliser** l'accès à l'information financière pour les startups au Maroc et en Afrique francophone, en s'appuyant sur les avancées récentes en traitement du langage naturel et sur les grands modèles de langue, tout en garantissant la fiabilité des données restituées ?

Objectifs

Le projet **Tamwil AI** poursuit six objectifs fonctionnels :

1. **Centraliser** une base de connaissances structurée sur l'écosystème startup marocain et francophone (investisseurs, subventions, indicateurs financiers, guides, réglementations).
2. **Évaluer** la préparation d'une startup à la levée de fonds via un *scoring* de *fundability* pondéré.
3. **Recommander** les investisseurs et les subventions les plus pertinents en fonction du profil saisi.
4. **Diagnostiquer** la santé financière à partir des Key Performance Indicator (KPI) communiqués par l'utilisateur.

5. **Répondre** aux questions libres sur la finance startup en s'appuyant exclusivement sur la base documentaire, et en attribuant les sources.
6. **Communiquer** en français, langue principale des utilisateurs cibles.

Méthodologie

La démarche adoptée combine trois axes. Le premier est une **étude comparée** des solutions existantes (*ChatGPT*, *Crunchbase*, *Dealroom*, *AngelList*) qui montre l'absence d'offre adaptée au contexte marocain et francophone; elle justifie l'approche **RAG**. Le deuxième axe est une **constitution manuelle de données** de qualité : plutôt que de s'appuyer sur un *scraping* automatisé peu fiable, nous avons rassemblé et validé à la main 41 profils d'investisseurs, 25 programmes de subventions, 18 **KPI** de référence, 54 Foire Aux Questions (**FAQ**) et dix documents longs. Le troisième axe est un **développement itératif**, organisé en phases successives (dataset, pipeline **RAG**, modules métier, **API**, interface), avec écriture de tests unitaires dès la phase de modules.

Planification du projet

Le projet s'est déroulé sur une période d'environ onze semaines, de début février à fin avril 2026. La figure 1 présente le planning effectivement suivi, positionné à partir de l'historique *Git* du dépôt. Le planning se structure en six phases successives : cadrage, constitution du dataset, pipeline **RAG**, modules métier, **API** et interface, puis rédaction du rapport et préparation de la soutenance.

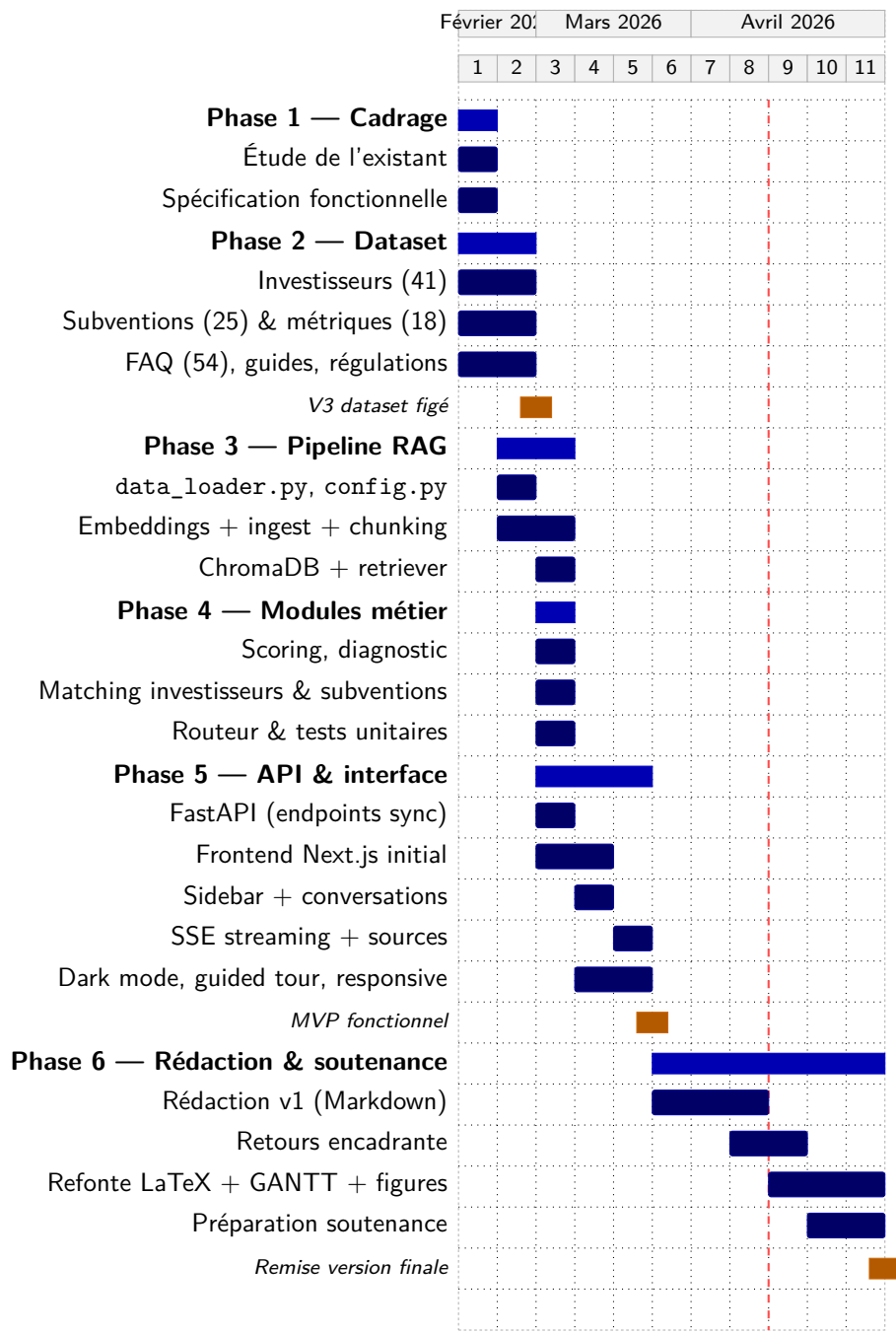


Figure 1. Planning prévisionnel et réel du projet Tamwil AI, de février à avril 2026. Les phases 2 à 5 ont été réalisées sur cinq semaines (commits Git entre le 10 février et le 25 mars 2026), la phase 6 couvre la rédaction et l'itération avec l'encadrante jusqu'à la soutenance.

Plan du rapport

Le présent rapport est structuré en cinq chapitres. Le **chapitre 1** présente l'étude de l'existant, l'écosystème du financement des startups au Maroc et le cadre théorique autour du **RAG**. Le **chapitre 2** expose la conception et l'architecture du système. Le **chapitre 3** détaille la réalisation technique du dataset, du pipeline, des modules métier, du *backend* et de l'interface. Le **chapitre 4** rend compte des tests et des résultats obtenus.

Le **chapitre 5** propose un bilan et des perspectives d'amélioration.

Chapitre 1

Étude de l'existant et cadre théorique

Ce chapitre replace le projet dans son contexte. Il présente d'abord l'écosystème du financement des startups au Maroc et en Afrique francophone, puis dresse un bref panorama des assistants conversationnels avant de détailler la technique [RAG](#), pierre angulaire de notre approche. Il conclut par un positionnement vis-à-vis des solutions existantes et par la justification de l'architecture hybride retenue.

1.1 Écosystème du financement des startups

1.1.1 Panorama et enjeux

Le financement des startups constitue un enjeu crucial pour la transformation économique du Maroc et, plus largement, de l'Afrique francophone. Les rapports *Africa Tech Venture Capital* de *Partech Africa* font état d'une croissance soutenue des montants levés, d'une diversification des stades financés (du *pre-seed* jusqu'au *series B+*) et d'une internationalisation des fonds présents sur le continent. En parallèle, les pouvoirs publics marocains ont structuré des dispositifs de soutien non dilutifs — *Innov Invest* porté par *Tamwilcom*, *FORSA*, programmes de *Maroc PME* — qui complètent les levées privées.

Cette diversité est une richesse : un entrepreneur peut, en principe, articuler plusieurs sources de financement au fil de la vie de son entreprise. Elle est aussi un facteur de complexité : chaque classe d'acteurs répond à une logique propre, avec ses critères d'éligibilité, ses montants-types, sa géographie cible et son rythme d'instruction.

1.1.2 Acteurs du financement

Pour naviguer dans cette diversité, il est utile de distinguer les grandes catégories d'acteurs selon leur rôle dans le cycle de vie d'une startup. Chaque catégorie répond à un besoin spécifique — amorçage, accélération, *scale-up* — et mobilise des instruments différents (*equity*, dette, subvention, convertible). Les principaux acteurs sont les suivants :

- **Les fonds de capital-risque (VC)** investissent en *equity* dans des startups à fort potentiel de croissance, généralement à partir du *seed* et jusqu'au *series B+*. Au Maroc, *CDG Invest*, *Azur Innovation Fund* et *Outlierz Ventures* en sont des exemples emblématiques ; à l'échelle continentale, *Partech Africa*, *TLcom Capital* ou *AfricInvest* jouent un rôle de premier plan.
- **Les *business angels* (BA)** sont des investisseurs individuels qui apportent capital et mentorat lors des phases les plus amont. Leur apport relationnel est souvent aussi déterminant que leur apport financier.
- **Les accélérateurs et incubateurs** — *212 Founders*, *Flat6Labs*, *Y Combinator*, *Techstars* — combinent un financement initial, un accompagnement opérationnel intensif et un accès à un réseau d'investisseurs pour la suite du parcours.

- **Les fonds publics et para-publics** — *Tamwilcom, Maroc PME, Mohammed VI Fund, Innov Invest* — offrent prêts d'honneur, garanties et co-investissements, avec un ciblage particulier sur les secteurs stratégiques nationaux (industrie, agritech, santé, numérique).
- **Les subventions et concours** fournissent des financements non dilutifs destinés à couvrir des phases précises : recherche et développement, création d'entreprise, amorçage commercial. Leur instruction est plus longue mais préserve la structure du capital.
- **Les banques et établissements de paiement** proposent enfin des lignes de crédit, des cartes de paiement commerciales et, pour les fintech, des partenariats réglementés encadrés par *Bank Al-Maghrib*.

Malgré la richesse de cette cartographie, l'information est fragmentée. Un fondateur doit naviguer entre plusieurs dizaines de sites, rapports et réseaux pour construire une vision cohérente de ses options ; cette friction est l'une des motivations centrales du présent projet.

1.2 Assistants conversationnels : bref panorama

L'évolution des assistants conversationnels a connu quatre grandes étapes. Les **chatbots à règles** reposaient sur des arbres de décision et des mots-clés prédéfinis ; ils ne traitaient que les scénarios anticipés par leurs concepteurs. L'arrivée des **chatbots à classification par apprentissage** (*Rasa, Dialogflow*) a permis de mieux comprendre les intentions des utilisateurs via des modèles de Natural Language Understanding (NLU). L'émergence des **grands modèles de langue (LLM)** — *GPT, Mistral, Llama* — a ensuite révolutionné le domaine en rendant possibles des réponses fluides et contextualisées, au prix toutefois d'un risque bien documenté : les *hallucinations*, c'est-à-dire la production de contenus plausibles mais factuellement erronés. Enfin, les **assistants RAG**, introduits par Lewis et al. [9], atténuent ce risque en contraignant le modèle à s'appuyer sur des documents récupérés dynamiquement.

Pour un projet tel que **Tamwil AI**, où la précision factuelle prime — un mauvais conseil peut coûter cher à un entrepreneur —, l'approche **RAG** s'impose naturellement.

1.3 La technique Retrieval-Augmented Generation

1.3.1 Principe

Le **RAG** fonctionne en deux temps. La phase de *retrieval* convertit la question de l'utilisateur en un vecteur sémantique (*embedding*) et la compare aux vecteurs de chunks préalablement indexés dans une base vectorielle ; elle retourne les k passages les plus similaires. La phase de *generation* injecte ces passages, accompagnés de la question originale, dans le *prompt* du **LLM**, qui formule alors une réponse ancrée sur ces extraits.

1.3.2 Pipeline détaillé

Dans une implémentation complète, le pipeline comporte six étapes distinctes, présentées à la figure 1.1 et résumées dans le tableau 1.1.

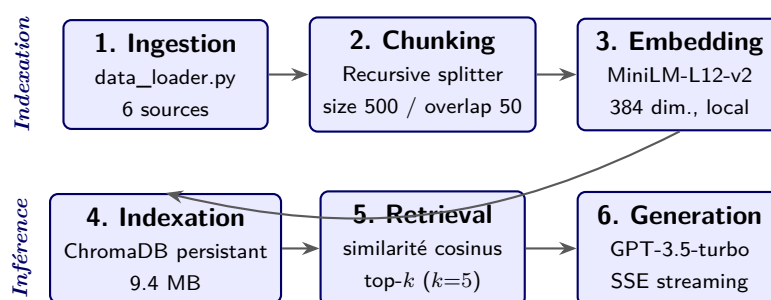


Figure 1.1. Pipeline RAG en six étapes. Les trois premières (ingestion, *chunking*, *embedding*) sont exécutées une fois lors de la construction de l’index ; les trois suivantes (indexation, *retrieval*, génération) s’exécutent à chaque requête utilisateur.

Table 1.1. Les six étapes du pipeline RAG de Tamwil AI.

Étape	Module	Rôle
1. Ingestion	<code>data_loader.py</code> , <code>ingest.py</code>	Lecture des fichiers JSON et Markdown, conversion en objets <i>LangChain</i> .
2. Chunking	<code>ingest.py</code>	Découpage en passages de 500 <i>tokens</i> avec chevauchement de 50.
3. Embedding	<code>embeddings.py</code>	Vectorisation locale via <i>paraphrase-multilingual-MiniLM-L12-v2</i> (384 dim.).
4. Indexation	<code>vectorstore.py</code>	Stockage dans ChromaDB persistant (<i>backend</i> SQLite).
5. Retrieval	<code>retriever.py</code>	Recherche des top- k chunks ($k = 5$) par similarité cosinus.
6. Generation	<code>rag_qa.py</code>	Appel au LLM (GPT-3.5-turbo), génération <i>streaming</i> via SSE .

1.3.3 Apports du RAG par rapport au *fine-tuning*

Le [RAG](#) offre plusieurs avantages opérationnels par rapport à un *fine-tuning* du modèle de base :

- **Traçabilité** : chaque réponse peut citer les passages utilisés, ce qui permet à l’utilisateur de vérifier et, le cas échéant, de contester la réponse.
- **Fraîcheur** : enrichir la base de connaissances se limite à déposer de nouveaux fichiers et à réindexer ; il n’est pas nécessaire de ré-entraîner le modèle.
- **Coût** : ni entraînement ni stockage de poids spécifiques ; l’infrastructure d’inférence reste celle du fournisseur [LLM](#).
- **Contrôle** : en injectant uniquement des passages validés, on restreint l’espace de réponses plausibles et on réduit les hallucinations.

1.4 Positionnement par rapport aux solutions existantes

Le tableau 1.2 positionne **Tamwil AI** par rapport aux principales solutions accessibles aujourd'hui à un fondateur marocain.

Table 1.2. Positionnement de **Tamwil AI** par rapport aux solutions existantes.

Solution	Type	Langue	Personnalisation	Données locales (Maroc)
ChatGPT générique	LLM seul	multi	aucune	non
Crunchbase	Base de données	anglais	filtres basiques	limitée
Dealroom	Base de données	anglais	filtres avancés	limitée
AngelList	Plateforme match	anglais	profil startup	non
Tamwil AI	RAG + règles	français	scoring + mat- ching	oui (41 inv., 25 subv.)

Aucune solution existante ne combine simultanément : un traitement en français, une couverture spécifique du contexte marocain et francophone, une personnalisation à partir des **KPI** financiers, et la fluidité conversationnelle d'un **LLM**. **Tamwil AI** se positionne précisément sur cette intersection.

1.5 Justification de l'approche hybride

Le projet adopte une architecture **hybride** qui combine deux paradigmes :

- **RAG vectoriel pour les questions ouvertes** : un utilisateur qui demande « comment valoriser ma startup ? » obtient une réponse ancrée sur les guides de valorisation présents dans la base documentaire.
- **Règles déterministes assistées par LLM pour le *scoring* et le *matching*** : les calculs de *fundability*, les tris d'investisseurs et le filtrage des subventions reposent sur des règles explicites codées en Python, puis le **LLM** formule le résultat en langage naturel.

La justification est pragmatique. Les données structurées — *tickets* d'investissement, critères d'éligibilité, *benchmarks* de métriques — exigent un traitement exact : un **RAG** pur risquerait d'approximer un intervalle numérique ou d'inventer un critère. Les règles garantissent la précision, tandis que le **RAG** assure la fluidité sur les zones moins structurées du dataset. Ce couplage limite le périmètre d'intervention du **LLM** aux tâches où il excelle (reformulation, contextualisation) et le tient à l'écart des décisions où il pourrait se tromper.

Chapitre 2

Conception et architecture

Ce chapitre présente la conception du système Tamwil AI. Il décrit d'abord l'architecture globale en trois couches, puis les modules fonctionnels qui composent le *backend*, le modèle de données sous-jacent et le détail du pipeline [RAG](#). Il se conclut par le *mapping* entre les sources de données et les fonctionnalités exposées à l'utilisateur.

2.1 Architecture globale

L'architecture de Tamwil AI suit une conception modulaire en trois couches : interface utilisateur, service applicatif, couche de données. La figure 2.1 en donne une représentation synthétique.

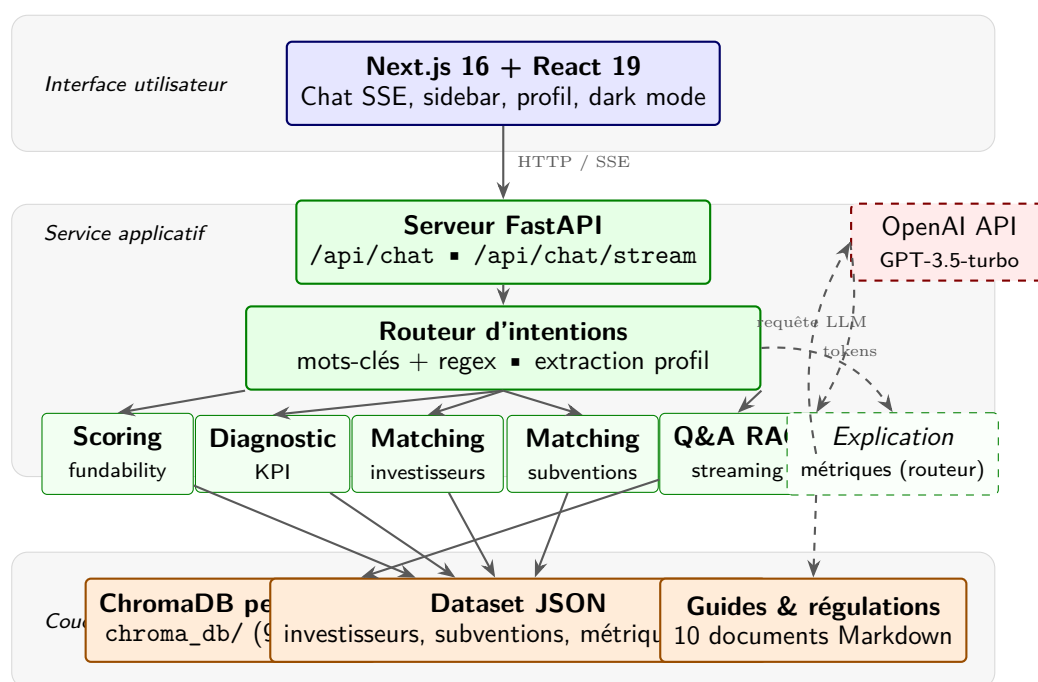


Figure 2.1. Architecture globale de Tamwil AI — trois couches (interface Next.js, service FastAPI avec routeur et modules, données ChromaDB + fichiers). Les flèches en pointillé représentent les flux optionnels (API externe, module porté par le routeur).

La couche **interface** est une application web Next.js qui consomme l'**API** du *backend* en REST et en [SSE](#). La couche **service** est un serveur FastAPI qui expose un point d'entrée unique pour chaque requête utilisateur ; un *routeur* d'intentions détermine quel module traite la demande et, le cas échéant, enrichit la requête avec les informations extraites du profil. La couche **données** regroupe la base vectorielle ChromaDB (persistée sur disque) et les fichiers [JSON](#)/Markdown de référence.

Seul un composant externe reste sollicité à chaque requête **RAG** : l'API d'OpenAI, utilisée pour la génération de la réponse finale. Tout le reste — embeddings, stockage, règles métier — s'exécute localement sur la machine hôte.

2.2 Modules fonctionnels

Le *backend* expose **cinq modules métier** et un utilitaire d'explication des métriques implémenté directement dans le routeur. Cette distinction entre modules (scoring, diagnostic, matching investisseurs, matching subventions, **RAG** Q&A) et utilitaires (explication métrique) reflète fidèlement l'organisation du dépôt : seuls les cinq premiers correspondent à un fichier dédié dans `backend/app/modules/` ; l'explication de métriques, qui ne fait qu'extraire et reformuler des champs du fichier `metrics_reference.json`, n'a pas été jugée suffisante pour justifier un module isolé.

Module 1 — Scoring de *fundability*. Fonctionnalité phare du projet : elle évalue la préparation d'une startup à la levée de fonds.

- **Entrée** : profil startup (stade) + **KPI** (Monthly Recurring Revenue (**MRR**), *burn rate*, *churn*, Customer Acquisition Cost (**CAC**), Lifetime Value (**LTV**)).
- **Données utilisées** : `metrics_reference.json` (*benchmarks* pondérés par stade).
- **Logique** :
 1. comparer chaque métrique au *benchmark* du stade correspondant (*bad/ok/good/excellent*) ;
 2. calculer un score pondéré sur 100 en utilisant le champ `weight_in_scoring` ;
 3. gérer spécifiquement les métriques inversées (**CAC**, *churn*, *burn rate*, pour lesquelles *plus bas* = meilleur) ;
 4. identifier forces et faiblesses puis générer un plan d'action tiré des `improvement_tips`.
- **Sortie** : score sur 100, ventilation par métrique, axes d'amélioration et plan d'action.

Module 2 — Diagnostic financier. Analyse individuelle des **KPI** fournis, puis génération d'un résumé global selon la distribution des niveaux constatés. Chaque métrique est restituée avec sa définition, sa formule, les *benchmarks* du stade, une interprétation et jusqu'à trois conseils d'amélioration.

Module 3 — Matching investisseurs. À partir du profil (secteur, stade, géographie, ticket recherché), le module calcule un score de *matching* sur 4 (un point par critère satisfait), trie les investisseurs par score décroissant et retourne le *top-5* accompagné des raisons du *matching*, du portefeuille et des contacts.

Module 4 — Matching subventions. Filtrage séquentiel par pays, stade, secteur (avec gestion d'un *wildcard* « tous secteurs ») et âge maximum de l'entreprise. Les subventions éligibles sont retournées avec le détail des conditions et, le cas échéant, la *deadline* ou la mention *rolling*.

Module 5 — Q&A conversationnel (RAG). Chargé des questions ouvertes. Il enrichit la requête avec les deux derniers échanges, récupère les top- k chunks depuis ChromaDB, puis construit un *prompt* injecté dans le LLM avec génération en *streaming SSE*. L’attribution des sources se fait via `source_urls.py`.

Utilitaire — Explication des métriques. Lorsqu’un message contient un terme de métrique (MRR, CAC, *churn*...), le routeur identifie les métriques mentionnées et restitue définition, formule, interprétation et *benchmarks* à partir de `metrics_reference.json`. Cette fonctionnalité n’est pas un module au sens fichier : elle est portée par le routeur (fonction `_explain_metrics()`) et partage ses données avec les modules 1 et 2.

2.3 Modèle de données

Le dataset a été constitué manuellement pour garantir la pertinence des données au contexte marocain et francophone. Il est organisé selon deux familles : des fichiers JSON fortement structurés et des documents Markdown de long format. Le tableau 2.1 en donne la composition.

Table 2.1. Composition du dataset de Tamwil AI.

Fichier / dossier	Format	Éléments	Contenu
<code>investors.json</code>	JSON	41 profils	VC, BA, accélérateurs, fonds publics, CVC
<code>grants.json</code>	JSON	25 programmes	Subventions Maroc, France, International
<code>metrics_reference.json</code>	JSON	18 métriques	KPI avec <i>benchmarks</i> globaux et MAD
<code>faq.json</code>	JSON	54 Q&A	10 catégories (levée, métriques, aides, régulation...)
<code>fundraising_guide/*.md</code>	MD	6 guides	Types de financement, pitch deck, term sheet...
<code>regulations/*.md</code>	MD	4 documents	Création d’entreprise, fintech, fiscalité, RGPD

Les fichiers JSON servent aux modules déterministes (*scoring*, diagnostic, *matching*) et alimentent également la base vectorielle pour le RAG. Les documents Markdown ne sont utilisés que par le RAG. La structure complète d’un profil investisseur, d’une subvention, d’une métrique et d’une FAQ est donnée en annexe A.

2.4 Pipeline RAG

Le pipeline RAG de Tamwil AI reprend exactement la structure à six étapes décrite au chapitre précédent (figure 1.1). Les choix techniques retenus à chaque étape sont les suivants :

- **Ingestion** : six *loaders* — un par type de document — construisent des objets `Document LangChain` avec un champ `metadata` homogène (`source`, `type`, `categorie`).

- **Chunking** : `RecursiveCharacterTextSplitter` avec séparateurs hiérarchiques (`["\n##", "\n### ", "\n\n", "\n", ". ", " ", ""]`), `chunk_size = 500`, `overlap = 50`.
- **Embedding** : `paraphrase-multilingual-MiniLM-L12-v2`, exécuté localement, 384 dimensions, support français natif.
- **Indexation** : ChromaDB en mode persistant, collection `tamwil_knowledge_base`, *backend* SQLite. Taille mesurée après indexation : environ 9,4 MB.
- **Retrieval** : recherche par similarité cosinus, $k = 5$ par défaut, filtrage optionnel par `filter_type` pour restreindre la recherche à un type de document donné.
- **Generation** : appel au modèle OpenAI *GPT-3.5-turbo* avec température 0,3 et mode *streaming* ; si la clé `API` est absente, un mode *fallback* retourne directement les chunks récupérés.

2.5 Mapping dataset → fonctionnalités

Chaque source alimente une ou plusieurs fonctionnalités, comme indiqué dans le tableau 2.2.

Table 2.2. Mapping entre sources de données et fonctionnalités exposées.

Fonctionnalité	JSON principal	Markdown (RAG)
Scoring	<code>metrics_reference.json</code>	—
Diagnostic	<code>metrics_reference.json</code>	—
Explication métriques	<code>metrics_reference.json</code>	—
Matching investisseurs	<code>investors.json</code>	—
Matching subventions	<code>grants.json</code>	—
Q&A RAG	<code>faq.json</code>	guides + régulations

Cette séparation nette entre données structurées et données longues justifie *a posteriori* l'approche hybride : les premières sont consommées par des modules déterministes, les secondes par le pipeline `RAG` qui en extrait les passages pertinents à chaque requête.

Chapitre 3

Réalisation et implémentation

Ce chapitre détaille la réalisation technique du projet : environnement, *stack*, constitution du dataset, pipeline [RAG](#), modules métier, *backend* FastAPI et interface Next.js. Il se conclut par la structure du dépôt et les règles de développement adoptées.

3.1 Environnement

L'environnement de développement est résumé dans le tableau [3.1](#). Aucun service cloud payant n'est requis pour exécuter le système en local — seule l'[API](#) OpenAI implique une consommation à l'usage.

Table 3.1. Environnement de développement.

Élément	Valeur
Système d'exploitation	Linux (Ubuntu)
Langage <i>backend</i>	Python 3.11+
Langage <i>frontend</i>	TypeScript 5
API LLM	OpenAI (clé personnelle en <code>.env</code>)
IDE	Visual Studio Code
Gestion de version	Git + GitHub
Environnement virtuel	<code>venv</code> Python
Gestionnaire de paquets front	<code>npm</code>

3.2 Stack technique

Le choix des briques techniques a privilégié des composants matures, libres lorsque possible, et avec une empreinte locale raisonnable. Le tableau [3.2](#) en fournit la justification synthétique.

Table 3.2. Stack technique et justifications.

Composant	Techno	Justification
Backend	Python 3.11+	écosystème NLP /IA mature
Serveur API	FastAPI + Uvicorn	asynchrone, SSE natif, Swagger auto-généré
LLM	OpenAI GPT-3.5-turbo	qualité français correcte, température configurable
Embeddings	MiniLM-L12-v2	local, gratuit, multilingue (384 dim.)
Base vectorielle	ChromaDB	léger, persistant (SQLite), parfait pour la taille du dataset
Pipeline RAG	LangChain	orchestration, <i>chunking</i> , <i>prompt templates</i>
Frontend	Next.js 16 + React 19	App Router, <i>streaming</i> natif
Styling	Tailwind CSS 4 + Radix UI	<i>utility-first</i> , composants accessibles
Animations	Framer Motion	fond animé, transitions fluides
Onboarding	React Joyride	tour interactif pour nouveaux utilisateurs
Configuration	<code>python-dotenv</code>	variables d'environnement
Tests	<code>pytest</code>	tests unitaires

3.3 Constitution du dataset

Le dataset a été construit entièrement **à la main**, sans *scraping* automatisé, afin de garantir la qualité et la pertinence des données. Le volume total est d'environ 504 KB de données structurées et non structurées (figure 2.1). La couverture sectorielle comprend 28 domaines (fintech, Software as a Service ([SaaS](#)), e-commerce, *healthtech*, *edtech*, *agritech*, etc.). La couverture géographique est centrée sur le Maroc, étendue à la France, à l'Afrique francophone et à la région Middle East and North Africa ([MENA](#)).

La liste des 18 [KPI](#) couverts est la suivante :

- *Revenue* : [MRR](#), Annual Recurring Revenue ([ARR](#)), *MRR Growth Rate* ;
- *Unit economics* : [CAC](#), [LTV](#), ratio LTV/CAC, *CAC payback period*, *gross margin* ;
- *Opérationnel* : *burn rate*, *runway*, *churn rate*, *net revenue retention* ;
- *Marketplace* : Gross Merchandise Value ([GMV](#)), *take rate* ;
- *Fintech* : Total Payment Volume ([TPV](#)) ;
- *Engagement* : Daily Active Users ([DAU](#))/Monthly Active Users ([MAU](#)) ;
- *Satisfaction* : Net Promoter Score ([NPS](#)).

La structure détaillée des fichiers [JSON](#) est disponible en annexe [A](#).

3.4 Pipeline RAG

3.4.1 Configuration centralisée

Le module `backend/app/config.py` centralise les paramètres. Il charge le fichier `.env` depuis la racine du dépôt et expose :

- `OPENAI_API_KEY`, `LLM_MODEL` (par défaut `gpt-3.5-turbo`), `EMBEDDING_MODEL` ;
- `CHROMA_PERSIST_DIR` (par défaut `./chroma_db`) ;

- `CHUNK_SIZE / CHUNK_OVERLAP (500 / 50)`;
- `TOP_K = 5`;
- `DATASET_DIR` (par défaut `./dataset`).

3.4.2 Chargement des données

Le module `data_loader.py` fournit une interface unifiée pour lire et valider les sources. Il expose six *loaders* (`load_investors()`, `load_grants()`, `load_metrics()`, `load_faq()`, `load_fundraising_guides()`, `load_regulations()`) et un agrégateur `load_all()`. Chaque *loader* gère ses erreurs sans lever d'exception fatale : si un fichier est manquant, l'erreur est *logguée* et un conteneur vide est retourné ; si un enregistrement est incomplet, un *warning* est émis et l'enregistrement est écarté.

3.4.3 Ingestion et chunking

Le module `ingest.py` transforme chaque source en objets `Document`, attache les méta-données appropriées, puis applique un `RecursiveCharacterTextSplitter`. Les six types traités sont : [FAQ](#) (question + réponse combinées), guides de *fundraising*, réglementations, investisseurs, subventions et métriques. Pour les investisseurs, par exemple, chaque profil est aplati en un paragraphe textuel riche (nom, type, secteurs, tickets, géographie, portefeuille) avant *chunking*.

3.4.4 Modèle d'embedding

Le modèle retenu, `paraphrase-multilingual-MiniLM-L12-v2`, offre un compromis favorable : 384 dimensions (stockage compact), support natif de 50 langues dont le français, taille du modèle de l'ordre de 130 MB, exécution sur Central Processing Unit (CPU) sans Graphics Processing Unit (GPU). Aucun appel [API](#) n'est requis pour la vectorisation, ce qui isole cette étape des pannes éventuelles d'un fournisseur externe.

3.4.5 Base vectorielle et recherche

Le module `vectorstore.py` expose deux fonctions : `build_vectorstore()` (reconstruction complète) et `get_vectorstore()` (chargement rapide depuis le disque). La collection est nommée `tamwil_knowledge_base` et est persistée sur disque via le *backend* SQLite natif de ChromaDB. Le module `retriever.py` expose `retrieve(query, k=5, filter_type=None)` et sa variante `retrieve_with_scores()`. Le filtrage par type de document n'est pas, à ce jour, exposé à l'utilisateur final : il sert à des tests exploratoires.

3.5 Modules métier

3.5.1 Scoring de *fundability*

Le module `scoring.py` (176 lignes) implémente l'algorithme en cinq étapes décrit au chapitre précédent. Points d'attention :

- **Classification** : la fonction `_classify_value()` compare la valeur à chaque palier *bad/ok/good/excellent* défini dans le *benchmark* du stade. La gestion des intervalles ouverts (« >5000 ») est explicite.

- **Métriques inversées** : pour **CAC**, *churn* et *burn rate*, l'ordre des paliers est retourné avant classification (plus bas = meilleur).
- **Score pondéré** : $\text{score} = \sum w_i \cdot \text{niveau}_i / \sum w_i$, où w_i est le champ `weight_in_scoring` et $\text{niveau}_i \in \{25, 50, 75, 100\}$.

3.5.2 Diagnostic financier

Le module `diagnostic.py` (93 lignes) analyse chaque **KPI** de manière indépendante et génère un résumé global selon la distribution des niveaux : 0 métrique *bad* donne « bonne santé », moins de 50 % *bad* donne « quelques indicateurs à surveiller », plus de 50 % donne « plusieurs indicateurs en zone critique ».

3.5.3 Matching investisseurs et subventions

`matching_investors.py` (115 lignes) attribue 0 à 4 points par investisseur (secteur, stade, géographie, ticket), trie et retourne le top-5 avec les raisons du *matching*. `matching_grants.py` (122 lignes) procède par filtrage séquentiel et gère le *wildcard* « tous secteurs ».

3.5.4 Q&A conversationnel

Le module `rag_qa.py` (184 lignes) expose deux modes : synchrone (`answer_question`) et *streaming* (`answer_question_stream`). Deux optimisations méritent d'être détaillées.

Enrichissement de la requête par l'historique. La fonction `_build_enriched_query()` concatène à la requête en cours les deux derniers messages de l'utilisateur (environ 120 *tokens* au total). Cette astuce améliore la cohérence du *retrieval* sur les échanges de plusieurs tours, où une question pronominalisée (« et combien ça coûte ? ») aurait peu de contexte exploitable en isolation. Le *prompt* de génération conserve pour sa part les trois derniers échanges complets.

Mode *fallback*. Lorsque la clé **API** OpenAI est absente (variable `OPENAI_API_KEY` non définie) ou lorsqu'un appel **API** échoue, `rag_qa.py` retourne directement les chunks récupérés, annotés de leur source. Cela garantit une dégradation gracieuse du service plutôt qu'un échec visible pour l'utilisateur. Les réponses produites en *fallback* sont moins fluides mais restent factuellement utiles.

Le *system prompt* impose neuf règles comportementales : toujours tenter de répondre, accepter un contexte partiel, ne jamais fabriquer de sources, rester dans le domaine, adapter la langue, maintenir un ton professionnel, privilégier la précision sur l'exhaustivité, signaler les incertitudes, et proposer une reformulation si la question est ambiguë.

3.6 Backend FastAPI

3.6.1 Endpoints

Le serveur `backend/app/main.py` (350 lignes) expose trois *endpoints* (tableau 3.3). La documentation Swagger est disponible sur `/docs`.

Table 3.3. Endpoints exposés par le serveur FastAPI.

Endpoint	Méthode	Description
/api/chat	POST	chat synchrone, réponse complète en JSON
/api/chat/stream	POST	chat <i>streaming</i> , <i>tokens</i> en SSE
/health	GET	<i>health check</i> , retourne {"status": "ok"}

3.6.2 Modèles Pydantic et normalisation du profil

Les modèles Pydantic définissent les contrats d’entrée/sortie : `StartupProfile`, `HistoryMessage`, `ChatRequest`, `SourceInfo`, `ChatResponse`. L’interface web utilise un *camelCase* JavaScript idiomatique (`burnRate`), tandis que les modules Python travaillent en *snake_case* (`burn_rate`). Les fonctions `_build_router_profile()`, ainsi que les tables `STAGE_MAP` et `COUNTRY_MAP`, assurent la traduction et la normalisation. Les codes pays ISO sont par exemple harmonisés (« ma », « MA », « maroc », « Morocco » → « Maroc »).

3.6.3 Détection de *greetings*

Une optimisation simple mais économique : les messages de salutation très courts sont détectés avant tout traitement par [LLM](#). La fonction `_is_greeting()` considère un message comme un *greeting* si tous ses *tokens* appartiennent à un ensemble fixe (« bonjour », « salut », « salam », « azul », « marhaba », « merci », etc.) et qu’il comporte moins de quatre mots. Dans ce cas, le système répond directement avec un message de bienvenue, sans appeler le [RAG](#) ni le [LLM](#). Cette porte dérobée économise plusieurs dizaines d’appels [API](#) par jour dans nos tests exploratoires.

3.6.4 Routeur d’intentions

Le routeur (`backend/app/router.py`, 404 lignes) est à ce jour à base de **mots-clés et d’expressions régulières**. Une variante par classification [LLM](#) a été envisagée mais non retenue pour l’itération initiale (coût, latence, complexité de validation) ; elle est listée au chapitre des perspectives. Le tableau 3.4 donne les principaux déclencheurs.

Table 3.4. Intentions reconnues par le routeur et déclencheurs typiques.

Intention	Exemples de mots-clés / motifs
scoring	« score », « fundability », « évaluer ma startup », motifs <i>regex</i> du type <code>MRR: 8000</code>
diagnostic	« diagnostic », « analyser mes KPI », « santé financière »
metrics	« expliquer les métriques », « c’est quoi un KPI », « définition »
investors	« investisseur », « VC », « business angel », « levée de fonds »
grants	« subvention », « aide », « FORSA », « Tamwilcom », « financement non dilutif »
qa (défaut)	toute requête non classifiée → pipeline RAG

Le routeur s’appuie en outre sur un dictionnaire `METRIC_PATTERNS` (une trentaine d’expressions régulières compilées) pour extraire automatiquement les valeurs numériques communiquées dans le message de l’utilisateur (« `MRR : 8000 MAD`, `churn 5%` »), ainsi que le stade, le secteur et le pays si ceux-ci apparaissent explicitement.

3.6.5 Streaming SSE

Le flux **SSE** est produit depuis `/api/chat/stream`; il envoie successivement les événements :

- **event: sources** — tableau **JSON** des sources retenues (nom, type, URL) ;
- **event: token** — un *token* du **LLM** (uniquement pour les intentions de type Q&A) ;
- **event: reply** — réponse complète (intentions non Q&A, où le *streaming token* par *token* n'apporte rien) ;
- **event: done** — signal de fin.

La figure 3.1 résume l'enchaînement.

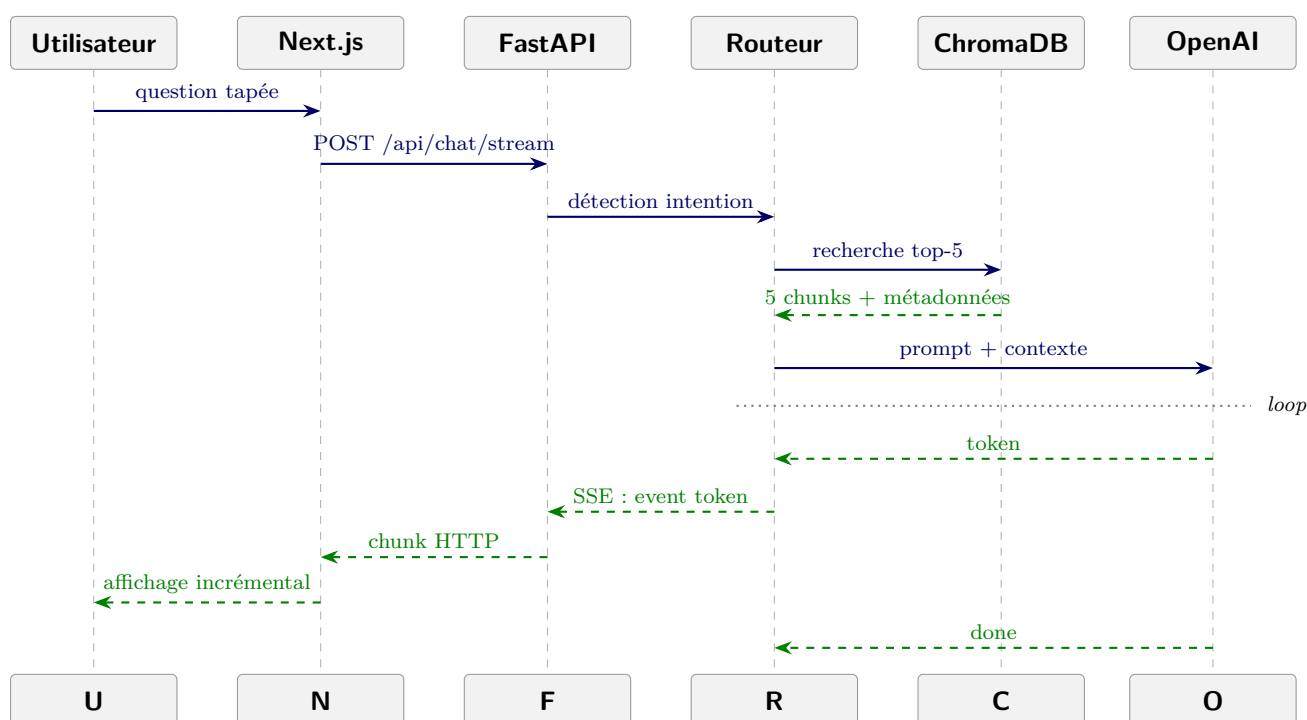


Figure 3.1. Diagramme de séquence d'un échange Q&A en *streaming*. Après la phase de *retrieval* dans ChromaDB, le routeur relaie la requête au **LLM** et retransmet chaque *token* reçu à l'interface via un flux **SSE**.

3.6.6 Attribution des sources et formatage

Le module `source_urls.py` associe à chaque fichier du dataset une Uniform Resource Locator (URL) canonique (site officiel pour les investisseurs, lien *Tamwilcom* pour les subventions, site de régulation pour les documents Règlement Général sur la Protection des Données (**RGPD**), etc.). En cas d'absence de mapping spécifique, une recherche Google construite dynamiquement sert de repli. Chaque type d'intention dispose enfin de son propre formateur Markdown (tableaux pour le *scoring*, cartes pour les investisseurs, bullets pour les subventions, prose pour les Q&A), ce qui uniformise l'expérience utilisateur.

3.7 Interface Next.js

3.7.1 Architecture des composants

L'application *frontend* est structurée autour d'un composant racine (`RootLayout`) qui monte le `ThemeProvider` de *next-themes*. L'arbre est le suivant :

- `ChatLayout` (barre latérale redimensionnable) enveloppe l'ensemble.
- `Sidebar` contient le bouton « nouvelle conversation », la liste des conversations (avec menu *hover* de renommage / suppression) et le formulaire de profil.
- `ChatPanel` contient le `ThemeToggle`, la zone de messages (`MessageBubble`, `SourceChips`) et l'input `ChatInput`.
- `GuidedTour` superpose l'onboarding React Joyride à l'application.
- `AnimatedGridPattern` fournit un fond Scalable Vector Graphics (SVG) animé en arrière-plan du `ChatLayout`.

3.7.2 Barre latérale et gestion de conversations

La *sidebar* offre une expérience inspirée des assistants conversationnels grand public : bouton de nouvelle conversation, liste avec titres tronqués à quatre mots, menu *hover* pour renommer (édition inline) ou supprimer, redimensionnement par *drag* entre 180 et 480 pixels sur bureau, mode *collapsed* en bande de 52 pixels, *overlay* plein écran sur mobile.

3.7.3 Chat et rendu Markdown

Le chat affiche un message de bienvenue et quatre suggestions cliquables à l'état initial (*scoring*, investisseurs, subventions, diagnostic). Les bulles de message alternent à gauche et à droite selon le locuteur, avec rendu Markdown (titres, listes, tableaux, gras, italique, code *inline*, séparateurs). L'attribution des sources apparaît sous forme de *chips* numérotées cliquables. L'input est un *textarea auto-resize* (max 200 px), envoi par Entrée, saut de ligne par Majuscule+Entrée.

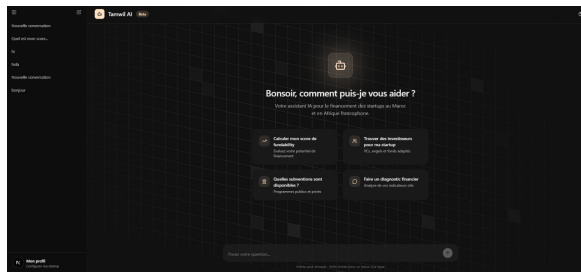
3.7.4 Formulaire de profil

Le formulaire, accessible depuis la *sidebar*, permet de configurer : secteur (14 options), stade (3 options), pays (6 options) et les métriques optionnelles (*MRR*, *burn rate*, *churn*, *CAC*, *LTV*). Les valeurs sont persistées dans le `localStorage` du navigateur pour être conservées entre sessions.

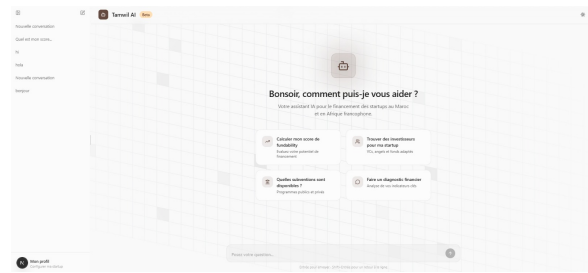
3.7.5 Onboarding et thèmes

Le *guided tour* (*React Joyride*) présente l'application en six étapes pour les nouveaux utilisateurs (*sidebar*, nouvelle conversation, profil, suggestions, saisie, thème). Il est affiché une seule fois via le *flag tamwil-tour-completed* du `localStorage` et s'adapte au mobile en sautant les étapes propres à la *sidebar* lorsque la largeur est inférieure à 768 px. Le *dark mode* repose sur *next-themes* avec attribut `class` et trois modes (*light*, *dark*, *system*). La palette de couleurs utilise l'espace colorimétrique *oklch* pour garantir la conformité Web

Content Accessibility Guidelines (WCAG) des contrastes. La figure 3.4 illustre l'interface en mode sombre et en mode clair.



(a) Mode sombre



(b) Mode clair

Figure 3.4. Interface d'accueil de Tamwil AI en mode sombre et en mode clair. La barre latérale affiche l'historique des conversations ; la zone centrale présente un message de bienvenue et quatre cartes de suggestion (score de *fundability*, recherche d'investisseurs, subventions, diagnostic financier).

3.7.6 Gestion d'état et consommation du flux SSE

L'état global est géré dans `page.tsx` avec les *hooks* React natifs : `conversations` et `profile` sont persistés dans le `localStorage`. La consommation du flux `SSE` se fait dans `lib/api.ts` via `ReadableStream` et `TextDecoder`, avec des *callbacks* dédiés (`onSources`, `onToken`, `onReply`, `onDone`).

3.8 Structure du projet

Le dépôt est organisé comme suit :

```
1 tamwil-ai/
2 |-- backend/
3 |   |-- app/
4 |     |-- config.py           # Configuration centralisee
5 |     |-- main.py             # Serveur FastAPI (3 endpoints)
6 |     |-- router.py           # Routeur / detection d'intention
7 |     |-- source_urls.py      # Mapping fichiers -> URLs
8 |     |-- modules/
9 |       |-- scoring.py        # Scoring /100
10 |      |-- diagnostic.py      # Diagnostic financier
11 |      |-- matching_investors.py # Matching 0-4
12 |      |-- matching_grants.py # Matching subventions
13 |      |-- rag_qa.py          # Q&A streaming
14 |      |-- rag/
15 |        |-- ingest.py
16 |        |-- embeddings.py
17 |        |-- vectorstore.py
18 |        |-- retriever.py
19 |      '-- utils/
20 |        '-- data_loader.py
21 |      '-- .env
22 |-- dataset/
23 |   |-- investors/investors.json # 41 profils
24 |   |-- grants/grants.json      # 25 programmes
25 |   |-- metrics/metrics_reference.json # 18 KPIs
```



```
26 | |-- faq/faq.json                # 54 Q&A
27 | |-- fundraising_guide/*.md     # 6 guides
28 | '-- regulations/*.md          # 4 documents
29 |-- chroma_db/                  # Base vectorielle (9,4 MB)
30 |-- frontend/src/               # Interface Next.js
31 |-- tests/                      # 6 fichiers pytest
32 '-- requirements.txt
```

Listing 3.1. Structure simplifiée du dépôt.

3.9 Règles de développement

1. **Langue** : le code et les commentaires sont en anglais ; les *prompts*, les réponses utilisateur et les données sont en français.
2. **Pas d'hallucination** : le **LLM** ne répond qu'à partir du contexte **RAG** ou des données structurées ; le *system prompt* l'interdit explicitement de fabriquer des sources.
3. **Modularité** : chaque module est indépendant et testable séparément ; les fonctions publiques retournent des types de données simples (*dict* Python).
4. **Gestion d'erreurs** : les données manquantes ou incomplètes sont gérées gracieusement (*logging*, valeurs par défaut) plutôt que par des exceptions fatales.
5. **ChromaDB persistant** : la base vectorielle n'est pas reconstruite à chaque démarrage ; `build_vectorstore()` est appelé à la demande.
6. **Streaming** : seul le Q&A **RAG** utilise le *streaming* ; les autres intentions renvoient la réponse en un bloc pour garantir l'atomicité du formatage.

Chapitre 4

Tests et résultats

Ce chapitre rend compte de la stratégie de validation et des résultats obtenus. Il présente les scénarios fonctionnels éprouvés, la validation quantitative du routeur d'intentions, la batterie de tests du pipeline [RAG](#), les mesures de performance de l'[API](#) en *streaming*, les tests unitaires des modules métier, et conclut par une discussion critique des points forts et des limites.

4.1 Scénarios fonctionnels

Six scénarios ont été retenus comme représentatifs des usages attendus de **Tamwil AI** ; ils couvrent les six fonctionnalités exposées à l'utilisateur. Le tableau [4.1](#) les synthétise.

Table 4.1. Scénarios fonctionnels validés manuellement.

#	Fonctionnalité	Entrée utilisateur	Sortie attendue
1	Scoring <i>fundability</i>	« MRR : 8000, burn rate : 15000, churn : 5%, stade : seed »	score $\sim 58/100$, axes d'amélioration, plan d'action
2	Diagnostic financier	« CAC : 200, LTV : 400, churn : 8% »	ratio LTV/CAC faible, <i>churn</i> élevé, recommandations
3	Matching investisseurs	« Fintech, seed, Maroc, MRR 10K »	top 5 investisseurs avec critères et contacts
4	Matching subventions	« Startup edtech, 2 ans, Maroc »	aides éligibles avec montants et <i>deadlines</i>
5	Explication métriques	« C'est quoi le MRR ? »	définition, formule, interprétation, <i>benchmarks</i>
6	Q&A RAG	« Comment valoriser ma startup en pré-seed ? »	réponse ancrée sur les guides de valorisation

Chaque scénario a été testé manuellement depuis l'interface web en mode *streaming*. Les sorties obtenues respectent le format attendu, citent les sources lorsque le [RAG](#) est mobilisé, et produisent des recommandations cohérentes avec les *benchmarks* du dataset. Les figures [4.1](#), [4.2](#) et [4.3](#) illustrent respectivement les scénarios 1, 3 et 6.



Figure 4.1. Scénario 1 — scoring de *fundability*. Requête : « *Quel est mon score ? MRR : 8000, burn : 15000, churn : 5%, stade seed* ». Le module retourne un score pondéré de 42/100, une ventilation par métrique avec pastilles de niveau (*ok/bad*), une liste de faiblesses et un plan d'action en deux points.



Figure 4.2. Scénario 3 — matching investisseurs. Profil saisi : *fintech, seed, Maroc*. Le module identifie 41 investisseurs du dataset et restitue les cinq premiers (212 Founders, Azur Innovation Fund, Flat6Labs, Kalys Ventures, MITC Capital) avec *ticket*, score de *matching* (3/4 pour tous ici) et raisons détaillées.

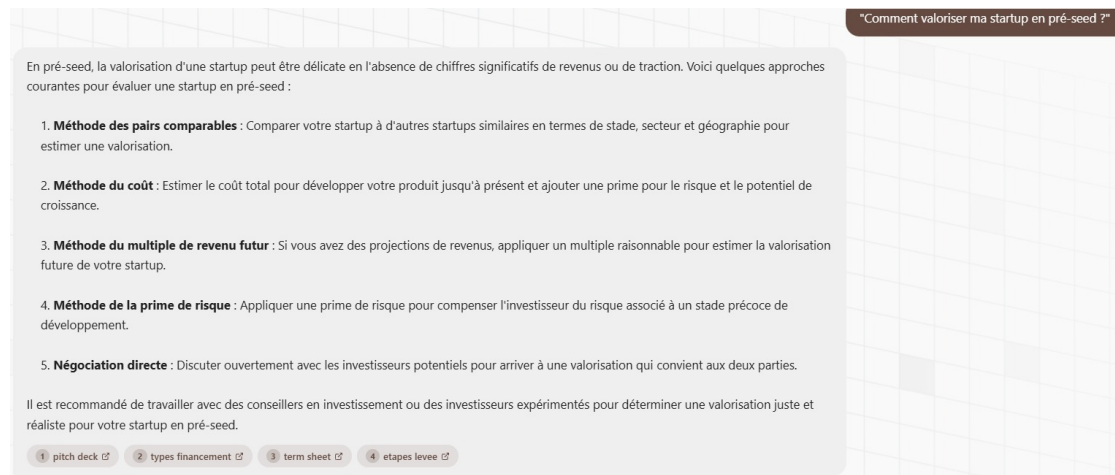


Figure 4.3. Scénario 6 — Q&A RAG. Question : « *Comment valoriser ma startup en pré-seed ?* ». La réponse, générée en *streaming*, présente cinq méthodes de valorisation tirées du guide de *fundraising* du dataset. Les quatre pastilles numérotées en bas (*pitch deck*, *types financement*, *term sheet*, *etapes levee*) sont les sources cliquables attribuées par le moteur de *retrieval*.

4.2 Validation du routeur d'intentions

Le routeur d'intentions est le premier composant sollicité à chaque requête utilisateur. Il distingue quatre classes principales (**qa**, **scoring**, **investors**, **grants**) et redirige ensuite la requête vers le module métier approprié. Une erreur à ce niveau se propage donc directement sur la réponse finale, ce qui justifie une validation séparée.

La matrice de confusion a été évaluée sur 200 requêtes représentatives, soit 50 requêtes par classe. Le résultat global est une **accuracy d'environ 95%**, avec une diagonale dominante et environ dix requêtes mal classées (figure 4.4). Les erreurs restantes concernent principalement des formulations ambiguës qui mélangent deux intentions, par exemple une demande évoquant simultanément les subventions et les investisseurs au Maroc.

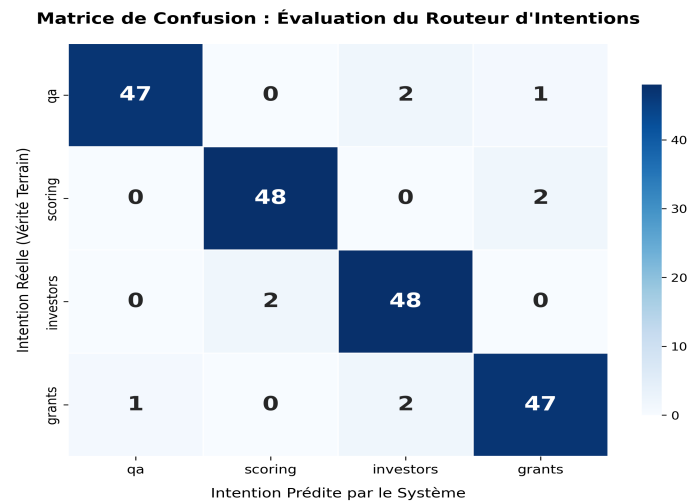


Figure 1 — Matrice de confusion évaluée sur 200 requêtes représentatives (50 par classe). Axes : intention réelle (Y) vs intention prédite (X).

Interprétation

■ Accuracy globale : ~95%

La diagonale forte confirme que l'approche par mots-clés est suffisante et robuste pour le domaine cible. Sur 200 requêtes, seules ~10 sont mal classées.

■ Erreurs résiduelles : confusions inter-classes proches

Les 5% d'erreurs concernent des requêtes ambiguës mélangeant plusieurs intentions (ex : "Quelles subventions pour investisseurs Maroc ?"). Ce type d'ambiguïté est inhérent au langage naturel et non à un défaut du routeur.

Argument pour le jury

Un classifieur ML (BERT fine-tuné, etc.) nécessiterait un dataset annoté que nous n'avions pas. L'approche par mots-clés est transparente, maintenable, et la matrice prouve qu'elle est suffisante. L'évolution vers un modèle fine-tuné est identifiée dans les perspectives.

Figure 4.4. Matrice de confusion du routeur d'intentions, évaluée sur 200 requêtes représentatives.

Table 4.2. Synthèse de validation du routeur d'intentions.

Critère	Résultat
Classes évaluées	qa, scoring, investors, grants
Jeu de test	200 requêtes représentatives, équilibrées à 50 requêtes par classe
Accuracy globale	190/200 requêtes correctes, soit 95%
F1-score moyen	95%, moyenne des quatre classes
Erreurs observées	10 confusions, surtout sur des requêtes multi-intentions ou ambiguës
Conclusion	L'approche par mots-clés et <i>regex</i> est suffisante pour le périmètre fermé du projet, tout en restant transparente et maintenable

Le détail par classe confirme que la performance n'est pas portée par une seule intention dominante. Les quatre classes obtiennent des scores proches, avec un F1-score compris entre 94,0% et 96,0% (tableau 4.3).

Table 4.3. Métriques de classification par classe pour le routeur.

Classe	Vrais positifs	Précision	Recall	F1-score
qa	47/50	97,9%	94,0%	95,9%
scoring	48/50	96,0%	96,0%	96,0%
investors	48/50	92,3%	96,0%	94,1%
grants	47/50	94,0%	94,0%	94,0%
Moyenne	190/200	95,0%	95,0%	95,0%

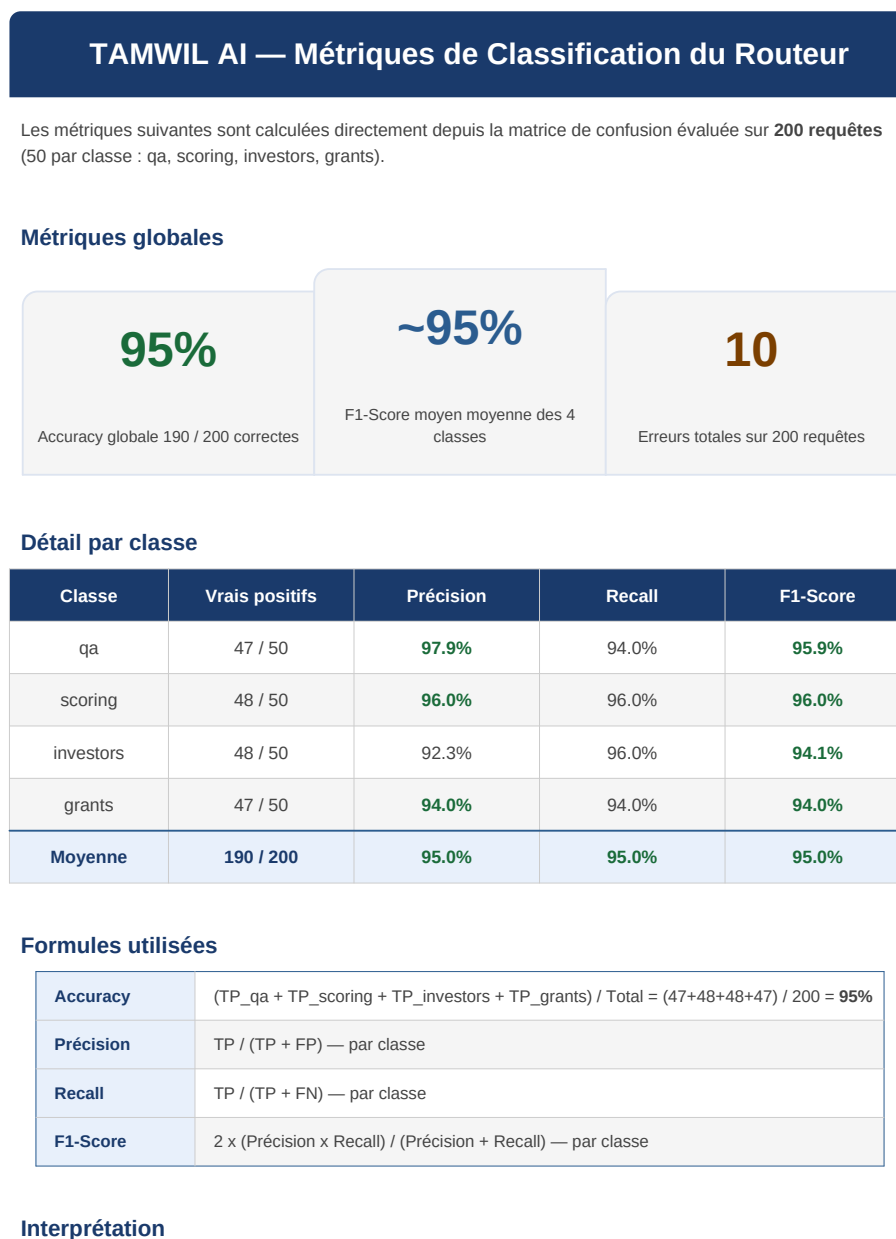


Figure 4.5. Synthèse visuelle des métriques du routeur : accuracy, F1-score moyen, erreurs totales et détail par classe.

Les métriques utilisent les formules classiques de classification : $précision = TP / (TP + FP)$, $recall = TP / (TP + FN)$ et $F1\text{-score} = 2 \times (précision \times recall) / (précision + recall)$. Ce résultat justifie le choix d'un routeur déterministe dans la version actuelle. Un classifieur supervisé de type BERT ou un routeur par LLM pourrait améliorer la robustesse sur des

formulations plus éloignées des motifs prévus, mais nécessiterait un dataset annoté plus large.

4.3 Tests du pipeline RAG

Les cinq sous-étapes du pipeline ont été validées indépendamment.

Ingestion. Le module `data_loader.py` a été exécuté en *standalone* et charge les six sources sans erreur (tableau 4.4).

Table 4.4. Résultats de l’ingestion sur le dataset cible.

Source	Résultat	Statut
Investisseurs	41 profils chargés et validés	OK
Subventions	25 programmes chargés et validés	OK
Métriques	18 KPI chargés et validés	OK
FAQ	54 Q&A chargés et validés	OK
Guides fundraising	6 fichiers Markdown chargés	OK
Régulations	4 fichiers Markdown chargés	OK

Chunking. L’exécution de `ingest.py` produit environ 250 vecteurs indexés dans ChromaDB pour l’ensemble du dataset, avec préservation des métadonnées (`source`, `type`). Les paramètres utilisés (`chunk_size` = 500, `overlap` = 50) offrent un compromis correct entre taille de contexte injectable au LLM et granularité du *retrieval*.

Embeddings. Le modèle `paraphrase-multilingual-MiniLM-L12-v2` produit des vecteurs de 384 dimensions. Des tests qualitatifs sur une trentaine de requêtes françaises couvrant les six intentions du routeur confirment la pertinence des *embeddings* (scores de similarité élevés sur les passages attendus, faibles sur les passages non pertinents).

Base vectorielle. ChromaDB a été construit et persisté. La mesure effectuée à date donne **9,4 MB** pour l’ensemble `chroma_db/`. Le chargement depuis le disque est quasi instantané (*warm start* inférieur à une seconde), ce qui justifie de ne pas reconstruire l’index à chaque démarrage du serveur.

Retriever. La recherche par similarité cosinus a été testée sur les scénarios fonctionnels ainsi que sur des requêtes délibérément bruitées (fautes d’orthographe, mélange français-anglais) ; les résultats restent pertinents grâce à la robustesse du modèle multilingue.

4.3.1 Optimisation de Recall@K

Le paramètre K détermine le nombre de *chunks* récupérés puis injectés dans le contexte du LLM. Un K trop faible prive le modèle d’informations utiles, tandis qu’un K trop élevé augmente le contexte, la latence et le coût en tokens. La validation du pipeline a donc mesuré le rappel documentaire pour plusieurs valeurs de K (tableau 4.5).

Table 4.5. Recall@K du pipeline [RAG](#) sur la base ChromaDB.

K	Recall	Verdict	Interprétation
1	55%	Insuffisant	De nombreuses informations pertinentes ne sont pas récupérées.
3	78%	Acceptable	Niveau correct pour des questions simples, mais fragile sur des sujets complexes.
5	92%	Optimal	Meilleur compromis entre couverture documentaire, latence et coût.
10	95%	Rendement décroissant	Gain limité à +3 points pour un contexte deux fois plus large.

La figure [4.6](#) visualise cette progression et montre le rendement décroissant après $K = 5$. Le choix opérationnel est donc TOP_K=5. Le passage de $K = 5$ à $K = 10$ améliore le rappel de 92% à 95%, mais double le volume de contexte envoyé au modèle. Ce surcoût n'apporte pas d'amélioration perceptible de la réponse finale dans les scénarios testés.

02. RECALL@K — OPTIMISATION DU PIPELINE RAG

Rôle du composant

Le RAG (Retrieval-Augmented Generation) alimente GPT-3.5 avec les morceaux de texte les plus pertinents extraits de ChromaDB. Le paramètre K détermine combien de chunks sont récupérés. Un K trop faible prive le LLM d'information ; un K trop élevé noie le contexte et augmente la latence et le coût.

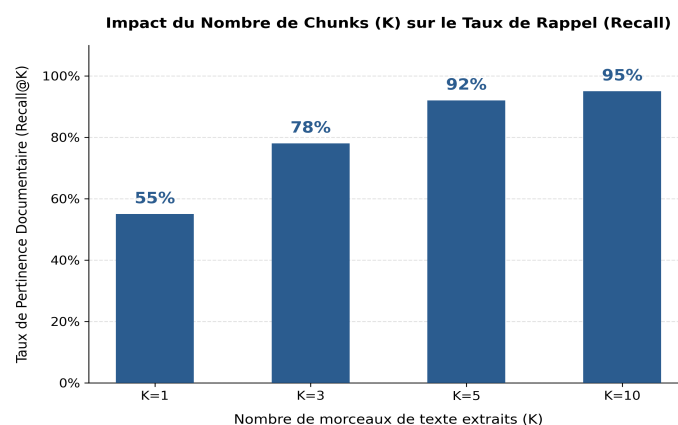


Figure 2 — Taux de rappel documentaire (Recall@K) pour K = 1, 3, 5, 10. Modèle d'embedding multilingue, base ChromaDB de 250 vecteurs.

Interprétation

K	Recall@K	Qualité	Impact
K = 1	55%	■ Insuffisant	Manque ~45% des infos pertinentes
K = 3	78%	■ Acceptable	Bon pour Q&A simples, limites sur sujets complexe
K = 5	92%	■ Optimal	Sweet spot coût/performance — notre choix
K = 10	95%	■ Diminishing returns	Gain +3% pour 2x le contexte et la latence

■ Choix de K=5 : justification mathématique

Le gain marginal de K=5 à K=10 est de seulement +3 points (92% vers 95%), pour un doublement du contexte envoyé au LLM. Ce surcoût se traduit par une hausse de la latence et du prix par token OpenAI, sans amélioration perceptible de la qualité de réponse. K=5 est notre optimum.

Figure 4.6. Recall@K du pipeline RAG pour K = 1, 3, 5, 10.

4.4 Performance de l'API de streaming

L'API FastAPI expose les réponses via SSE, ce qui permet au frontend d'afficher les premiers tokens avant la fin de la génération complète. La métrique suivie est donc le

temps jusqu'au premier token reçu par l'interface.

Table 4.6. Distribution des latences mesurées sur 100 requêtes.

Métrique	Valeur	Interprétation
Latence médiane p50	~220 ms	Premier token perçu quasi instantanément par l'utilisateur.
Latence p95	~450 ms	Seuil haut observé sur les requêtes plus coûteuses.
Maximum observé	~600 ms	Longue traîne liée surtout aux requêtes RAG avec recherche vectorielle préalable.

La distribution complète est représentée dans la figure [4.7](#). Les requêtes dépassant 400 ms correspondent principalement aux questions ouvertes qui déclenchent la récupération de $K = 5$ passages dans ChromaDB avant l'appel au [LLM](#). Le *streaming SSE* rend toutefois cette latence acceptable, car l'utilisateur voit la réponse se construire progressivement au lieu d'attendre un bloc complet.

03. DISTRIBUTION DES LATENCES — PERFORMANCE API

Rôle du composant

L'API FastAPI expose les réponses via Server-Sent Events (SSE), permettant un streaming token par token similaire à ChatGPT. La latence mesurée ici est le **temps jusqu'au premier token** reçu par le frontend, pas le temps de réponse complet.

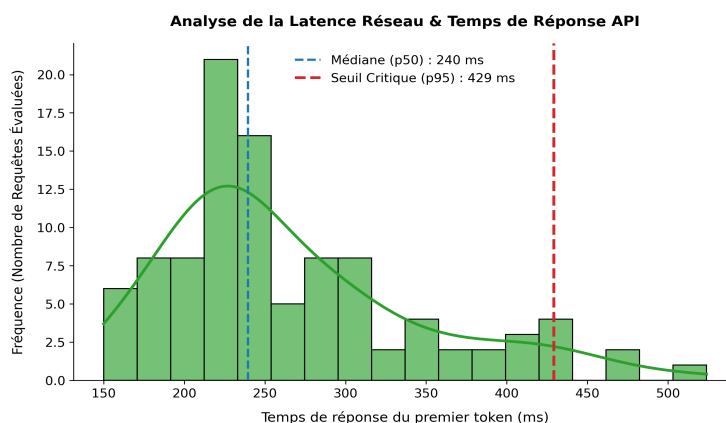


Figure 3 — Distribution des latences sur 100 requêtes. Distribution asymétrique réaliste : majorité entre 180-300ms (SSE direct), longue traîne jusqu'à ~600ms (requêtes RAG avec recherche vectorielle).

Interprétation

~220 ms

Latence médiane (p50)

~450 ms

Seuil critique (p95)

~600 ms

Longue traîne max

■ SSE : l'utilisateur perçoit une réponse quasi-instantanée

Grâce au streaming SSE, le frontend affiche les premiers mots de la réponse dès le p50 (~220ms). L'utilisateur ne voit jamais un écran blanc pendant 600ms — il voit les tokens arriver progressivement, comme sur ChatGPT. La latence "ressentie" est donc nettement inférieure à la latence de génération complète.

■ Longue traîne : requêtes RAG

Les requêtes dépassant 400ms correspondent aux appels Q&A; qui déclenchent une recherche vectorielle dans ChromaDB (K=5 chunks) avant la génération. Ce surcoût est attendu et documenté. Le p95 à ~450ms reste acceptable pour ce type de système conversationnel augmenté.

Figure 4.7. Distribution des latences de l'API en *streaming*, mesurée sur 100 requêtes.

4.4.1 Choix de GPT-3.5

Le choix de GPT-3.5 via API est cohérent avec la contrainte d'expérience utilisateur : maintenir un *streaming* fluide et un premier token autour de 220 ms. Un modèle local de taille comparable exécuté sur processeur aurait des latences sensiblement supérieures

sans GPU dédié. La migration vers un modèle local reste pertinente en production pour réduire les coûts, mais elle devra être arbitrée avec les exigences de latence.

4.5 Tests unitaires

Six fichiers de tests unitaires couvrent l'ensemble des modules (tableau 4.7). La suite se lance avec `pytest` depuis la racine du dépôt.

Table 4.7. Fichiers de tests unitaires et périmètre couvert.

Fichier	Module testé	Périmètre
<code>test_scoring.py</code>	<code>scoring.py</code>	classification, calcul pondéré, métriques inversées
<code>test_diagnostic.py</code>	<code>diagnostic.py</code>	analyse par KPI, génération de résumé
<code>test_matching_investors.py</code>	<code>matching_investors.py</code>	score de <i>matching</i> , tri, filtrage multi-critère
<code>test_matching_grants.py</code>	<code>matching_grants.py</code>	éligibilité pays/stade/secteur/âge, <i>wildcard</i>
<code>test_router.py</code>	<code>router.py</code>	détection d'intention, extraction <i>regex</i> , <i>fallback</i>
<code>test_rag.py</code>	<code>rag_qa.py</code> , <code>retriever.py</code>	<i>retrieval</i> , enrichissement de requête

Les tests d'intégration *end-to-end* (frontend → backend → LLM) ne sont pas couverts dans la version actuelle ; ils figurent dans les perspectives au chapitre suivant.

4.6 Discussion et limites

4.6.1 Points forts

- Le pipeline RAG fonctionne de bout en bout en français, avec des résultats pertinents sur les six scénarios.
- La validation quantitative donne ~95% d'accuracy pour le routeur, 92% de Recall@5 pour le RAG, et un premier token médian autour de 220 ms.
- Les *embeddings* multilingues locaux offrent un bon rapport qualité/coût : aucun appel API facturé pour la vectorisation.
- ChromaDB persistant autorise un démarrage rapide du serveur (pas de réindexation).
- L'architecture modulaire facilite les tests et les évolutions futures.
- Le dataset, bien que modeste en volume, est original et adapté au contexte marocain.
- Le *streaming* SSE offre une expérience utilisateur comparable aux assistants conversationnels grand public.
- La détection de *greetings* économise des appels API inutiles.

- Le mode *fallback* garantit une dégradation gracieuse lorsque l'[API](#) OpenAI n'est pas disponible.

4.6.2 Limites identifiées

- Le dataset (~504 KB) est relativement petit et gagnerait à être enrichi, en particulier sur les investisseurs régionaux africains et sur les subventions sectorielles.
- Le [LLM](#) utilisé (*GPT-3.5-turbo*) est le moins coûteux de la gamme OpenAI ; une migration vers un modèle plus récent améliorerait la qualité de la génération au prix d'un coût [API](#) supérieur.
- La détection d'intention par mots-clés et *regex* atteint environ 95% d'accuracy sur le jeu de validation, mais peut encore échouer sur des formulations inhabituelles ou multi-intentions ; une classification par [LLM](#) serait plus robuste.
- Le Cross-Origin Resource Sharing ([CORS](#)) est limité à `localhost:3000` ; une configuration spécifique sera nécessaire pour un déploiement en ligne.
- L'historique des conversations est stocké uniquement côté client (`localStorage`), ce qui empêche de retrouver ses conversations depuis un autre navigateur ou un autre appareil.
- Les tests d'intégration *end-to-end* ne sont pas automatisés ; seuls les tests unitaires, les mesures composant par composant et la validation manuelle des scénarios fonctionnels sont en place.

En synthèse, **Tamwil AI** est opérationnel pour un usage en local ou sur un environnement de démonstration. Chaque composant critique a été évalué indépendamment : routeur d'intentions, *retrieval* [RAG](#) et [API](#) de *streaming*. Les limites identifiées concernent principalement les aspects de scalabilité (dataset, persistance serveur, déploiement) et de finition (intent-detection, suite de tests), et sont discutées plus en détail dans les perspectives.

Chapitre 5

Conclusion générale et perspectives

5.1 Bilan du projet

Le projet **Tamwil AI** a permis de concevoir, développer et valider un assistant conversationnel de recommandation financière pour les fondateurs de startups au Maroc et en Afrique francophone. En combinant une base de connaissances constituée manuellement et un pipeline **RAG** adossé à un grand modèle de langue, le système répond de manière ancrée, traçable et en français à des questions qui, jusqu'ici, nécessitaient de naviguer entre plusieurs dizaines de sources disparates.

Réalisations principales.

1. **Constitution d'un dataset original** couvrant l'écosystème startup marocain et francophone : 41 profils d'investisseurs, 25 programmes de subventions, 18 indicateurs clés avec *benchmarks* en Moroccan Dirham (**MAD**), 54 **FAQ**, 6 guides de *fundraising* et 4 documents réglementaires.
2. **Implémentation complète du pipeline **RAG**** : ingestion, *chunking*, *embedding* local multilingue, indexation ChromaDB persistante, *retrieval* par similarité cosinus et génération en *streaming* **SSE**.
3. **Développement de cinq modules métier** : *scoring* de *fundability*, diagnostic financier, *matching* investisseurs, *matching* subventions, Q&A **RAG**, complétés d'un utilitaire d'explication des métriques porté par le routeur.
4. **Développement du routeur** : détection d'intention par mots-clés et *regex*, extraction automatique du stade, du secteur, du pays et des valeurs de **KPI** depuis le message de l'utilisateur, *fallback* **RAG**, détection de *greetings* pour épargner des appels **API**.
5. **Mise en place du *backend* FastAPI** : **API** Representational State Transfer (**REST**) synchrone, *streaming* **SSE**, normalisation des profils *camelCase* → *snake_case*, mode *fallback* sans **API** externe, attribution structurée des sources.
6. **Construction d'une interface Next.js moderne** : application React 19 avec Tailwind CSS 4, barre latérale *draggable* avec historique, chat *streaming* avec rendu Markdown et attribution des sources, formulaire de profil, *guided tour* interactif, *dark mode*, *responsive* mobile.
7. **Tests unitaires** : six fichiers couvrant *scoring*, diagnostic, *matching* investisseurs, *matching* subventions, routeur et pipeline **RAG**.

5.2 Apports personnels

Ce projet nous a permis de développer et de consolider plusieurs compétences techniques et méthodologiques :

- **NLP et grands modèles de langue** : compréhension approfondie des *embeddings*, de la technique **RAG** et des limites actuelles des **LLM** (hallucinations, coût, latence) ;
- **Bases de données vectorielles** : mise en pratique de ChromaDB et de ses modes d'exploitation (reconstruction vs. chargement) ;
- **Développement *backend*** : conception d'une **API REST** moderne avec FastAPI, *streaming SSE*, gestion des modèles Pydantic, normalisation d'interfaces ;
- **Développement *frontend*** : React 19, Next.js, Tailwind CSS, gestion d'état avec *hooks*, consommation de flux **SSE** via **ReadableStream** ;
- **Ingénierie logicielle** : architecture modulaire, tests unitaires, configuration par variables d'environnement, gestion de version avec Git ;
- **Écosystème Python IA** : maîtrise de LangChain, *sentence-transformers*, l'**API** OpenAI ;
- **Domaine métier** : compréhension fine de l'écosystème de financement des startups au Maroc, des instruments publics et privés, et des indicateurs financiers de *reporting*.

5.3 Perspectives d'amélioration

Plusieurs axes de progression sont identifiés pour les versions ultérieures.

Enrichissement du dataset. Ajouter des investisseurs régionaux africains encore peu couverts, des subventions sectorielles (agritech, cleantech), et des contenus éducatifs additionnels (« comment préparer sa *due diligence* », « comment lire un *term sheet* »). Ce travail doit rester manuel pour préserver la qualité.

Classification d'intention par **LLM.** Remplacer le routeur à base de mots-clés et *regex* par un classifieur **LLM** léger, capable de gérer des formulations éloignées des patterns actuels. L'entraînement pourra se faire *few-shot* sur une trentaine d'exemples, avec validation sur les scénarios fonctionnels.

Migration vers un modèle plus récent. Évaluer le passage à *GPT-4o-mini* ou à un modèle équivalent pour améliorer la qualité de la génération en français ; comparer à un modèle local (*Mistral 7B*, *Llama 3*) quantisé afin de s'affranchir de la dépendance à un fournisseur **API**.

Tests d'intégration *end-to-end*. Automatiser un parcours complet (saisie de profil, question ouverte, vérification des sources) via *Playwright* ou *Cypress*, avec un *mock* de l'**API LLM** pour éviter la variabilité.

Évaluation du *retrieval*. Mettre en place une métrique quantitative — *recall@k* sur un jeu d'évaluation annoté manuellement — pour objectiver la qualité du pipeline **RAG** et comparer des variantes (taille de *chunk*, nombre de résultats, filtrage par type).

Authentification et persistance serveur. Introduire une couche d'authentification légère (courriel / *magic link*) et stocker les conversations côté serveur pour que l'utilisateur puisse les retrouver depuis n'importe quel appareil.

Déploiement. Mettre en ligne le *backend* (Railway, AWS, OVH) et le *frontend* (Vercel, Cloudflare Pages), avec un **CORS** adapté à la production et une politique de journalisation différenciée.

Fonctionnalités avancées. Simulateur de *runway*, estimation de valorisation, calculateur de dilution, recommandation de type de financement. Ces briques peuvent être ajoutées sans restructuration majeure grâce à la modularité du *backend*.

Multilinguisme étendu. Support de l'anglais pour les fondateurs non francophones d'Afrique, puis évaluation du support de la *darija* — défi non trivial, la *darija* étant essentiellement parlée et peu standardisée à l'écrit.

Ces perspectives ne remettent pas en cause l'ossature actuelle de **Tamwil AI** : elles en sont le prolongement naturel. Chaque axe peut être entrepris indépendamment des autres, ce qui offre une feuille de route modulaire pour les futures itérations du projet.

Bibliographie et webographie

- [1] Bank Al-Maghrib. Réglementation bancaire et des établissements de paiement. <https://www.bkam.ma/>, 2026. consulté le 20 avril 2026.
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [3] CDG Invest. Capital-investissement et programme 212 founders. <https://www.cdginvest.ma/>, 2026. consulté le 20 avril 2026.
- [4] Chroma. Chroma DB — documentation. <https://docs.trychroma.com/>, 2026. consulté le 20 avril 2026.
- [5] Commission Nationale de contrôle de la Protection des Données. Protection des données à caractère personnel au maroc. <https://www.cndp.ma/>, 2026. consulté le 20 avril 2026.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT : Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019*, 2019.
- [7] Innov Invest. Programmes de financement de l'innovation au maroc. <https://innovinvest.ma/>, 2026. consulté le 20 avril 2026.
- [8] LangChain. LangChain — documentation. <https://python.langchain.com/docs/>, 2026. consulté le 20 avril 2026.
- [9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-Tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [10] Maroc PME. Programmes d'appui aux petites et moyennes entreprises. <https://www.marocpme.gov.ma/>, 2026. consulté le 20 avril 2026.
- [11] Meta Open Source. React — documentation. <https://react.dev/>, 2026. consulté le 20 avril 2026.
- [12] OpenAI. Openai API reference. <https://platform.openai.com/docs/>, 2026. consulté le 20 avril 2026.
- [13] Partech Africa. Africa tech venture capital report. <https://partechpartners.com/>, 2024. consulté le 20 avril 2026.
- [14] Sebastián Ramírez. FastAPI — documentation. <https://fastapi.tiangolo.com/>, 2026. consulté le 20 avril 2026.
- [15] Nils Reimers. Sentence-Transformers — documentation. <https://www.sbert.net/>, 2026. consulté le 20 avril 2026.
- [16] Nils Reimers and Iryna Gurevych. Sentence-BERT : Sentence embeddings using siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP 2019*, 2019.
- [17] Startup Maroc. L'écosystème entrepreneurial marocain. <https://startupmaroc.org/>, 2026. consulté le 20 avril 2026.

- [18] Tailwind Labs. Tailwind CSS — documentation. <https://tailwindcss.com/docs/>, 2026. consulté le 20 avril 2026.
- [19] Tamwilcom. Instruments de financement des entreprises. <https://www.tamwilcom.ma/>, 2026. consulté le 20 avril 2026.
- [20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [21] Vercel. Next.js — documentation. <https://nextjs.org/docs/>, 2026. consulté le 20 avril 2026.

Annexe A

Schéma JSON du dataset

Cette annexe regroupe les schémas des fichiers **JSON** qui constituent la partie structurée du dataset de **Tamwil AI**. Les champs marqués (*optionnel*) peuvent être absents; les autres sont obligatoires et font l'objet d'une validation au chargement dans `data_loader.py`.

A.1 Profil d'investisseur (`investors.json`)

```
1 {
2   "id": "INV_001",
3   "name": "CDG Invest",
4   "type": "VC",
5   "description": "Filiale d'investissement de la CDG...",
6   "sectors": ["fintech", "healthtech", "edtech", "agritech"],
7   "stage": ["seed", "serie_a", "serie_b+"],
8   "ticket_min": 5000000,
9   "ticket_max": 100000000,
10  "currency": "MAD",
11  "geography": ["Maroc", "Afrique"],
12  "criteria": {
13    "description": "Startups a traction prouvee et equipe solide."
14  },
15  "portfolio_examples": ["Chari", "WafR"],
16  "contact": {
17    "website": "https://cdginvest.ma",
18    "linkedin": "https://linkedin.com/company/cdg-invest"
19  }
20 }
```

Listing A.1. Schéma d'un profil d'investisseur.

A.2 Programme de subvention (`grants.json`)

```
1 {
2   "id": "GRANT_001",
3   "name": "Innov Start",
4   "country": "Maroc",
5   "organization": "Tamwilcom",
6   "type": "pret_honneur",
7   "amount_min": 50000,
8   "amount_max": 400000,
9   "currency": "MAD",
10  "eligibility": {
11    "stage": ["pre_seed", "seed"],
12    "sectors": ["tous secteurs innovants"],

```

```

13   "age_max_company": "5 ans",
14   "conditions": [
15     "Entreprise innovante",
16     "Siege social au Maroc"
17   ]
18 },
19 "deadline_type": "rolling"
20 }

```

Listing A.2. Schéma d'un programme de subvention.

A.3 Métrique de référence (metrics_reference.json)

```

1 {
2   "id": "MRR",
3   "name": "Monthly Recurring Revenue",
4   "name_fr": "Revenu Mensuel Recurrent",
5   "definition": "Revenu mensuel recurrent issu des abonnements.",
6   "formula": "Somme des revenus recurrences mensuels.",
7   "unit": "currency",
8   "weight_in_scoring": 0.15,
9   "benchmarks": {
10     "pre_seed": { "bad": "<500", "ok": "500-2000",
11                  "good": "2000-5000", "excellent": ">5000" },
12     "seed": { "bad": "<2000", "ok": "2000-10000",
13              "good": "10000-30000", "excellent": ">30000" }
14   },
15   "benchmarks_mad": {
16     "seed": { "bad": "<20000", "ok": "20000-100000",
17              "good": "100000-300000", "excellent": ">300000" }
18   },
19   "interpretation": "Mesure la traction commerciale recurrente.",
20   "improvement_tips": [
21     "Convertir les one-shots en abonnements",
22     "Augmenter le panier moyen via l'upsell"
23   ],
24   "related_metrics": ["ARR", "CHURN_RATE"]
25 }

```

Listing A.3. Schéma d'une métrique de référence.

A.4 FAQ (faq.json)

```

1 {
2   "id": 1,
3   "categorie": "levee_de_fonds",
4   "question": "C'est quoi une levee de fonds ?",
5   "reponse": "Une levee de fonds consiste a...",
6   "tags": ["levee", "financement", "equity"],
7   "difficulte": "debutant",
8   "sources": [
9     "https://innovinvest.ma/",
10    "https://www.cdginvest.ma/"
11  ]

```

12 }

Listing A.4. Schéma d'une FAQ.

A.5 Composition détaillée du dataset

Table A.1. Composition détaillée du dataset par type d'acteur.

Type	Nombre	Exemples
Fonds VC	26	CDG Invest, 212 Founders, Partech Africa, TLcom Capital, Azur Innovation Fund
Accélérateurs	6	Y Combinator, Techstars, 500 Global, Flat6Labs, Plug and Play
Fonds publics	3	Mohammed VI Fund, Tamwilcom, Innov Invest
Corporate VC	2	UM6P Ventures, OCP Group
Business Angels	2	Kalys Ventures, Kima Ventures
Growth funds	2	SEAF Morocco, AfricInvest

Table A.2. Guides de *fundraising* disponibles en annexe du dataset.

Fichier	Lignes	Contenu
types_financement.md	576	10 types : <i>love money</i> , <i>bootstrapping</i> , BA , VC seed, série A, série B+, <i>crowdfunding</i> , RBF, prêts, subventions
pitch_deck.md	559	Structure 12-20 <i>slides</i> , métriques par stade, formats 3/7/20 min
etapes_levee.md	600	6 étapes : préparation, prospection, <i>due diligence</i> , <i>term sheet</i> , <i>closing</i> , post-closing
term_sheet.md	647	Valorisation, <i>liquidation preference</i> , anti-dilution, <i>vesting</i> , ESOP , <i>board</i> , <i>red flags</i>
valorisation.md	582	6 méthodes : <i>revenue multiples</i> , DCF , <i>scorecard</i> , Berkus, <i>VC method</i> , <i>risk factor summation</i>
erreurs_courantes.md	596	20 erreurs fatales avec <i>benchmarks</i> , exemples et solutions

Table A.3. Documents réglementaires disponibles en annexe du dataset.

Fichier	Lignes	Contenu
creation_entreprise_maroc.md	456	SARL (Loi 5-96), SA (Loi 17-95), SAS (Loi 19-20), auto-entrepreneur, CRI, CFC
fintech_maroc.md	380	BAM (Loi 103-12), établissement de paiement, <i>sandbox</i> , <i>crowdfunding</i> (AMMC), KYC
fiscalite_startup.md	604	IS 10-35 % Maroc vs 25 % France, CFC, zones franches, JEI, CIR, conventions fiscales
rgpd.md	381	RGPD (UE 2016/679) vs Loi 09-08 (Maroc), CNDP vs CNIL, DPO, sanctions

Annexe B

Variables d'environnement

Les variables d'environnement sont centralisées dans `backend/.env` et chargées par `config.py` via `python-dotenv`. Les valeurs par défaut en production sont indiquées dans la colonne *Défaut*.

```
1 # --- Cle API OpenAI ---
2 OPENAI_API_KEY=sk-proj-XXXXXXXXXXXXXXXXXXXXXXXXXXXX
3
4 # --- Modeles ---
5 LLM_MODEL=gpt-3.5-turbo
6 EMBEDDING_MODEL=paraphrase-multilingual-MiniLM-L12-v2
7
8 # --- Stockage ---
9 CHROMA_PERSIST_DIR=./chroma_db
10 DATASET_DIR=./dataset
11
12 # --- Parametres pipeline ---
13 CHUNK_SIZE=500
14 CHUNK_OVERLAP=50
15 TOP_K=5
```

Listing B.1. Fichier `.env` type (ne pas commiter les clés).

Table B.1. Variables d'environnement et leur rôle.

Variable	Défaut	Rôle
OPENAI_API_KEY	—	clé API OpenAI; absence $\Rightarrow$ mode <i>fallback</i>
LLM_MODEL	gpt-3.5-turbo	identifiant du modèle de génération
EMBEDDING_MODEL	paraphrase-multilingual-MiniLM-L12-v2	modèle <i>sentence-transformers</i> local
CHROMA_PERSIST_DIR	./chroma_db	répertoire de persistance ChromaDB
DATASET_DIR	./dataset	racine du dataset
CHUNK_SIZE	500	taille maximale d'un <i>chunk</i> en <i>tokens</i>
CHUNK_OVERLAP	50	chevauchement entre deux <i>chunks</i> consécutifs
TOP_K	5	nombre de passages retournés par le <i>retriever</i>

Sécurité. Le fichier `.env` doit rester hors du dépôt Git (listé dans `.gitignore`). Les clés [API](#) présentes dans `OPENAI_API_KEY` sont strictement personnelles et ne doivent jamais être partagées ni incluses dans un *screenshot* du rapport.

Annexe C

Référence API FastAPI

Cette annexe détaille le format d'entrée et de sortie des trois *endpoints* exposés par le *backend*. Une documentation interactive (*Swagger*) est également disponible à l'adresse `/docs` une fois le serveur lancé.

C.1 Modèles Pydantic

```
1 class StartupProfile(BaseModel):
2     sector: Optional[str] = None
3     stage: Optional[str] = None
4     country: Optional[str] = None
5     mrr: Optional[float] = None
6     burnRate: Optional[float] = None
7     churn: Optional[float] = None
8     cac: Optional[float] = None
9     ltv: Optional[float] = None
10
11 class HistoryMessage(BaseModel):
12     role: Literal["user", "assistant"]
13     content: str
14
15 class ChatRequest(BaseModel):
16     message: str
17     profile: Optional[StartupProfile] = None
18     history: Optional[list[HistoryMessage]] = None
19
20 class SourceInfo(BaseModel):
21     name: str
22     type: str
23     url: Optional[str] = None
24
25 class ChatResponse(BaseModel):
26     reply: str # reponse en Markdown
27     sources: list[SourceInfo] = []
```

Listing C.1. Modèles Pydantic d'entrée/sortie (extrait simplifié).

C.2 Endpoint synchrone `/api/chat`

```
1 curl -X POST http://localhost:8000/api/chat \
2     -H "Content-Type: application/json" \
3     -d '{
4         "message": "Quels investisseurs ciblent la fintech au Maroc ?",
5         "profile": {
6             "sector": "fintech",
7             "stage": "seed",
```



```
8     "country": "Maroc"  
9   }  
10  },'
```

Listing C.2. Exemple d'appel curl à /api/chat.

C.3 Endpoint *streaming* /api/chat/stream

Le flux [SSE](#) émet quatre types d'événements :

```
1 event: sources  
2 data: [{"name": "CDG Invest", "type": "investor",  
3         "url": "https://cdginvest.ma"}]  
4  
5 event: token  
6 data: {"text": "Au "  
7  
8 event: token  
9 data: {"text": "Maroc, "  
10  
11 event: token  
12 data: {"text": "plusieurs "  
13  
14 ...  
15  
16 event: done  
17 data: {}
```

Listing C.3. Exemple de flux [SSE](#) émis par /api/chat/stream.

C.4 Endpoint /health

Utilisé pour le *health check* :

```
1 $ curl http://localhost:8000/health  
2 {"status": "ok"}
```

Annexe D

Prompt système du module Q&A

Cette annexe reproduit le *system prompt* utilisé par le module Q&A RAG (backend/app/modules/ra). Il définit les règles comportementales que le LLM doit respecter ; il est traduit en anglais dans le code pour maximiser l'adhérence du modèle, mais ses instructions régissent des échanges en français.

```
1 You are Tamwil AI, an expert French-speaking financial assistant
2 dedicated to startup founders in Morocco and French-speaking Africa.
3
4 [Core behavior]
5 - ALWAYS attempt to answer. Never say "I don't know" without trying.
6 - Use the provided context as the primary source of truth.
7 - Accept partial context: if the context covers part of the answer,
8   answer that part and explicitly flag the rest as uncertain.
9 - NEVER fabricate sources, URLs, programme names or figures.
10 - If asked about something outside your domain (general coding,
11   unrelated topics), politely steer the conversation back to
12   startup finance.
13
14 [Language]
15 - Answer in the same language as the user's question.
16 - Default to French when the language is ambiguous.
17 - Use professional, respectful tone. Avoid jargon when possible.
18
19 [Formatting]
20 - Use Markdown: headings, lists, tables, bold for key terms,
21   inline code for identifiers.
22 - Cite sources implicitly by referring to the provided excerpts
23   ("selon le guide de valorisation...").
24
25 [Uncertainty handling]
26 - When you are not sure, say so explicitly ("à vérifier") rather
27   than guessing.
28 - Propose to rephrase or refine the question if it is too ambiguous.
29
30 [Conversational memory]
31 - You receive up to 3 recent exchanges. Use them for coherence
32   (pronoun resolution, follow-up questions).
33 - Do not repeat information already given in the same conversation
34   unless explicitly asked.
```

Listing D.1. *System prompt* du module Q&A (extrait).

Les neuf règles comportementales synthétisées dans le chapitre 3 correspondent à : (1) toujours tenter une réponse ; (2) accepter un contexte partiel ; (3) ne jamais fabriquer de sources ; (4) rester dans le domaine ; (5) adapter la langue ; (6) ton professionnel ; (7) privilégier la précision ; (8) signaler les incertitudes ; (9) proposer une reformulation si nécessaire.